Document d'Analyse - OnlyShoes

Équipe : Groupe_1_03

Membres de l'équipe (par ordre alphabétique) :

- COUSIN-GAUTIER
- GUIRADO-JESSY
- NAOUACH-HANNIBAL
- TERRIEN-SWAN

---

1. Approche

1.1 Présentation de l'équipe

Groupe : Groupe_1_03

Composition de l'équipe :

1. COUSIN-GAUTIER
2. GUIRADO-JESSY
3. NAOUACH-HANNIBAL
4. TERRIEN-SWAN

Organisation du travail :

L'équipe s'est organisée de manière collaborative en se répartissant les tâches :

- **Cahier des charges** : Réalisé par tout le groupe lors de l'analyse de l'énoncé de base
- **Modélisation UML** :
  - Cas d'utilisation : COUSIN-GAUTIER, TERRIEN-SWAN, GUIRADO-JESSY
  - Diagrammes d'activités : COUSIN-GAUTIER , TERRIEN-SWAN
  - Diagrammes de classes : TERRIEN-SWAN
- **Relecture et validation** : Chaque membre a relu le travail des autres, avec des discussions en cas de doute

Nous avons adopté une méthodologie itérative avec des reprises régulières du diagramme de classes d'analyse, des cas d'utilisation et des diagrammes d'activités pour assurer la cohérence entre les différentes vues.

1.2 Cahier des charges - Expression du besoin

Le cahier des charges est le point de départ de notre projet. Pour l'élaborer, nous avons adopté une approche originale : nous nous sommes mis dans la peau du client, comme dans un jeu de rôle, pour vraiment comprendre ses attentes et ses besoins. Cette démarche nous a permis de transformer l'énoncé fourni en un véritable cahier des charges structuré.

Comment nous avons procédé

La création du cahier des charges s'est faite en plusieurs étapes, chacune nous permettant d'affiner notre compréhension du projet :

1. **Première lecture : comprendre le projet**
    - Nous avons identifié qu'il s'agissait d'une marketplace de chaussures entre particuliers, un peu comme Vinted mais spécialisé
    - Nous avons repéré les différents acteurs : les clients qui achètent, les vendeurs qui proposent leurs chaussures, et les administrateurs qui veillent au bon fonctionnement
    - Nous avons listé toutes les fonctionnalités qui semblaient nécessaires
2. **Ensuite, on a creusé : formaliser les besoins**
    - Nous avons dégagé **35 exigences fonctionnelles** précises, que nous avons numérotées pour ne rien oublier
    - Pour y voir plus clair, nous les avons regroupées par thèmes : tout ce qui concerne les comptes utilisateurs, les annonces, la recherche, les avis, le paiement, l'administration et les signalements
    - Chaque exigence a été rédigée de façon claire pour éviter les malentendus
3. **Enfin, la mise en forme finale**
    - Description du domaine d'application pour poser le contexte
    - Liste complète et organisée de toutes les exigences
    - Prise en compte des contraintes légales (RGPD) et de sécurité

Ce qu'on en retient

Le cahier des charges que nous avons créé est devenu **notre boussole** pour tout le reste du projet. C'est ce document qui nous permet de vérifier qu'on est sur la bonne voie et que notre solution répondra bien à ce qui est attendu.

1.3 Démarche suivie pour identifier, collecter et formaliser les besoins fonctionnels

Sources complémentaires étudiées

Pour ne pas réinventer la roue et mieux comprendre le domaine, nous avons aussi regardé ce qui se fait ailleurs :

1. **Sites concurrents**
    - **Vinted** : pour voir comment fonctionne une marketplace entre particuliers qui marche bien
    - **Leboncoin** : notamment la section chaussures, pour comprendre comment les gens vendent leurs affaires en ligne

    - Nous avons noté les fonctionnalités qui marchent bien et les bonnes idées d'interface

2. **Documentation UML**
    - Le tutoriel UML de Helmo pour bien comprendre les cas d'utilisation
    - Les conventions PlantUML pour créer nos diagrammes de façon standard
    - Le site PlantUML pour la syntaxe et trouver des exemples

Notre façon de travailler

Nous avons travaillé de manière itérative et en équipe :

D'abord, analyser les besoins du cahier des charges

- Nous sommes repartis de nos 35 exigences fonctionnelles pour bien les comprendre
- Nous avons identifié qui fait quoi : visiteurs, clients connectés, vendeurs et administrateurs
- Nous avons organisé tout ça par grands domaines (comptes, annonces, recherche, etc.) pour y voir plus clair
- Nous avons relevé les contraintes techniques (comme le RGPD) à ne pas oublier

Ensuite, modéliser avec UML (en plusieurs passes)

- Nous avons créé les diagrammes de cas d'utilisation, domaine par domaine
- Nous avons détaillé les scénarios avec des diagrammes d'activités pour montrer comment les choses se déroulent
- Nous avons construit le diagramme de classes d'analyse pour structurer les données
- **Important** : nous avons fait plusieurs allers-retours entre ces différents diagrammes. À chaque fois qu'on trouvait une incohérence ou qu'un membre de l'équipe avait un retour, on reprenait pour améliorer

Enfin, valider ensemble

- Chacun a relu le travail des autres pour repérer les erreurs ou les oublis
- Quand il y avait un doute, on en discutait tous ensemble pour trancher
- Nous avons vérifié qu'on n'avait oublié aucune exigence du cahier des charges

---

1.4 Besoins non-fonctionnels identifiés

En plus des fonctionnalités visibles, nous avons pensé à tout ce qui rend l'application agréable et fiable à utiliser :

1.4.1 Performance et Efficacité

Pourquoi c'est important : Personne n'aime attendre devant un écran qui charge. Avec potentiellement des milliers d'annonces, il faut que ça reste rapide et fluide.

Ce qu'on a prévu :

- **Pages qui chargent vite** : on vise moins de 2 secondes pour afficher une page
- **Recherche instantanée** : résultats en moins d'1 seconde
- **Optimisation de la base de données** : des index sur les critères de recherche les plus courants (prix, taille, marque) et des procédures stockées pour les calculs complexes
- **Recherche intelligente** : grâce aux embeddings vectoriels (représentations mathématiques des descriptions), on peut trouver des chaussures similaires même si on utilise des mots différents. Par exemple, chercher "baskets rouges" trouvera aussi "sneakers écarlates"

1.4.2 Sécurité

Pourquoi c'est crucial : On manipule de l'argent et des données personnelles. La sécurité n'est pas négociable (c'est d'ailleurs dans l'exigence 27).

Ce qu'on a mis en place :

- **HTTPS partout** : toutes les communications sont chiffrées, impossible d'intercepter les données
- **Mots de passe protégés** : on utilise bcrypt, un algorithme de hachage très résistant aux attaques
- **Données sensibles chiffrées** : respect du RGPD avec un stockage ultra-sécurisé
- **Validation stricte** : on filtre toutes les saisies pour bloquer les tentatives de piratage (injections SQL, XSS...)
- **Sessions sécurisées** : déconnexion automatique après inactivité, renouvellement régulier des tokens
- **Paiements externalisés** : on ne stocke JAMAIS de données bancaires chez nous. Tout passe par des services certifiés comme Stripe ou PayPal (exigence 25)

1.4.3 Conformité RGPD

Pourquoi c'est obligatoire : C'est la loi, et c'est dans notre cahier des charges (exigences 10 et 11). Les utilisateurs ont des droits sur leurs données.

Comment on s'y conforme :

- **Droit à l'oubli** : un utilisateur peut demander la suppression totale de ses données (exigence 10). Tout est effacé, sans retour possible
- **Choix des consentements** : l'utilisateur choisit précisément ce qu'il accepte (cookies essentiels, cookies fonctionnels, traitement des données, communications marketing). On garde une trace de ses choix avec la date
- **Transparence totale** : dès l'inscription, on explique clairement ce qu'on fait avec les données
- **Export des données** : à tout moment, l'utilisateur peut récupérer ses données au format JSON
- **Pas de conservation éternelle** : après suppression d'un compte, l'historique des achats peut être anonymisé

1.4.4 Ergonomie et Accessibilité

Pourquoi on y tient : Une interface bien pensée, c'est ce qui fait qu'on aime utiliser un site (exigence 27). Et il faut que ça marche sur tous les écrans.

Nos choix :

- **Navigation simple** : on sait toujours où on est et comment aller ailleurs
- **Responsive** : que ce soit sur téléphone, tablette ou ordinateur, l'interface s'adapte automatiquement
- **Toujours un moyen de rentrer** : bouton "retour accueil" bien visible en haut de page (exigence 17)
- **Messages compréhensibles** : quand ça marche, quand ça plante, on le dit clairement
- **Accessible à tous** : respect des normes d'accessibilité (bon contraste, navigation au clavier possible, textes alternatifs pour les images)

1.4.5 Maintenabilité et Évolutivité

Pourquoi on y pense maintenant : Un code bien organisé, c'est du temps gagné plus tard pour corriger des bugs ou ajouter des fonctionnalités.

Notre organisation :

- **Architecture MVC** : on sépare bien les données, l'affichage et la logique grâce à CodeIgniter 4. Ça permet de modifier une partie sans casser le reste
- **Documentation** : les parties compliquées sont commentées pour qu'on s'y retrouve dans 6 mois
- **Tests automatisés** : les fonctionnalités critiques (paiement, recherche) sont testées automatiquement pour éviter les régressions
- **Procédures en base de données** : les calculs complexes sont dans PostgreSQL, centralisés et réutilisables
- **Patterns éprouvés** : on utilise des modèles de conception reconnus (DAO, Factory, Observer)

1.4.6 Fiabilité et Disponibilité

Pourquoi c'est vital : Imagine un paiement qui plante au milieu... L'enfer ! Il faut que tout soit cohérent et que ça marche tout le temps.

Nos garanties :

- **Transactions atomiques** : c'est tout ou rien ! Un paiement validé = annonce marquée vendue + commande créée + notification envoyée. Si une étape plante, tout est annulé
- **Logs détaillés** : on garde une trace de tout ce qui se passe (INFO, WARNING, ERROR) pour comprendre rapidement en cas de problème
- **Sauvegardes quotidiennes** : backup automatique de la base PostgreSQL tous les jours
- **Emails automatiques** : dès qu'une action importante se passe (commande, modification de compte...), un email part automatiquement (exigence 26)

1.4.7 Capacité et Scalabilité

- **Pagination** : pas question d'afficher 10 000 annonces d'un coup ! On découpe en pages

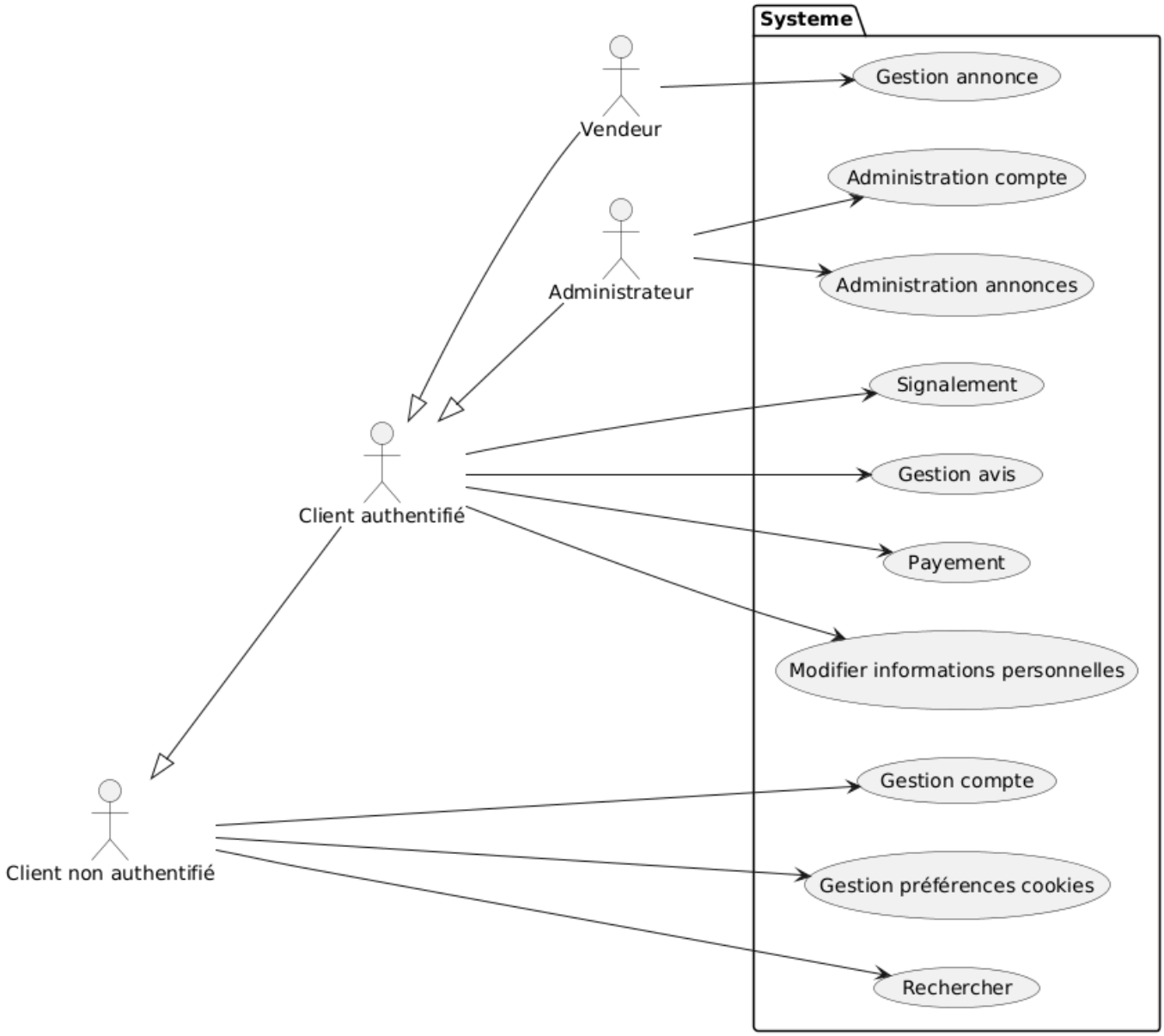
- **Mise en cache** : les données souvent consultées sont gardées en mémoire pour aller plus vite
- **Images optimisées** : compression et redimensionnement automatiques pour économiser de la bande passante
- **PostgreSQL** : une base de données robuste qui gère bien les requêtes complexes et les embeddings (avec l'extension pgvector)

---

2. Diagrammes de Cas d'Utilisation et Scénarios

2.1 Diagramme de Cas d'Utilisation Général

Le diagramme suivant présente une vue d'ensemble des principaux cas d'utilisation du système OnlyShoes, organisés par acteur.



Acteurs identifiés

1. **Client non authentifié** : Visiteur du site

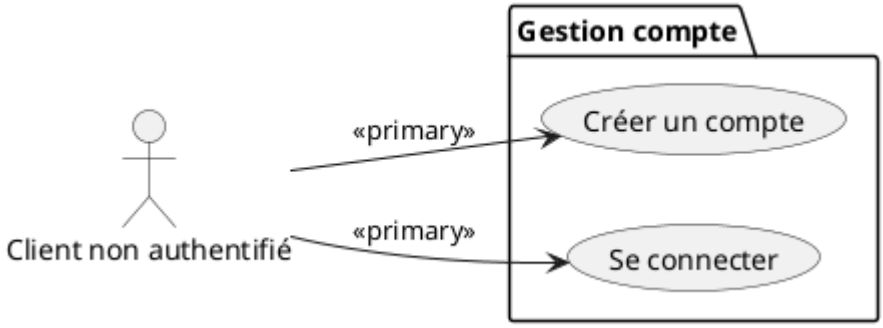
2. **Client authentifié** : Utilisateur connecté (acheteur)
3. **Vendeur** : Utilisateur authentifié publiant des annonces
4. **Administrateur** : Gestionnaire du système
5. **Système Bancaire** : Acteur externe pour les paiements

Cas d'utilisation principaux

Le système OnlyShoes propose les fonctionnalités suivantes, regroupées par domaine :

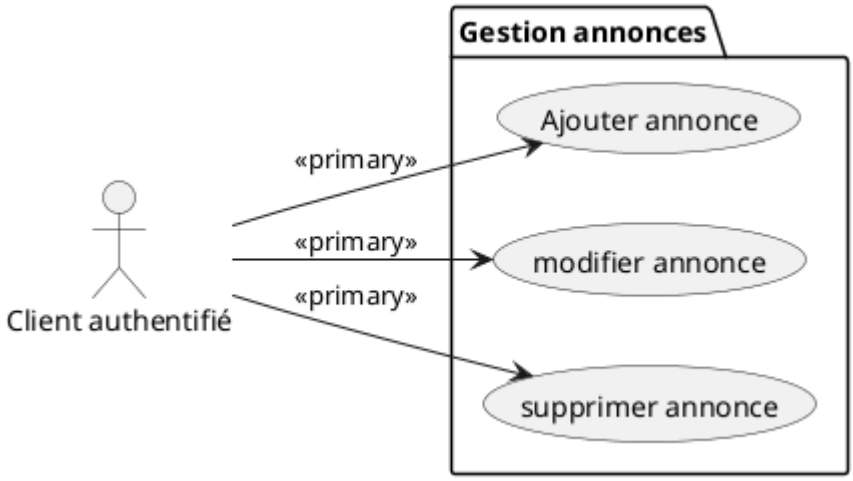
Gestion des comptes

- Créer un compte
- Se connecter
- Se déconnecter
- Modifier ses informations personnelles
- Supprimer son compte



Gestion des annonces

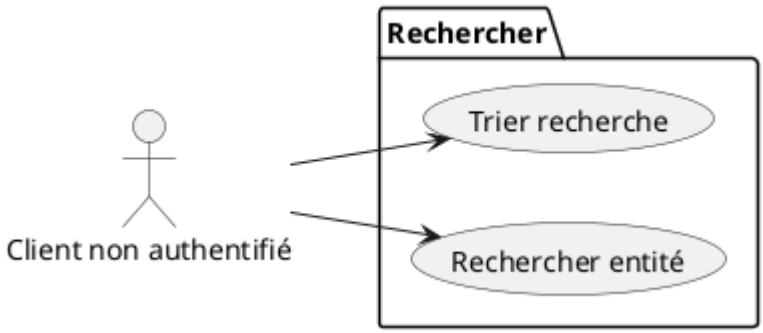
- Consulter les annonces
- Publier une annonce
- Modifier une annonce
- Supprimer une annonce



Recherche et navigation

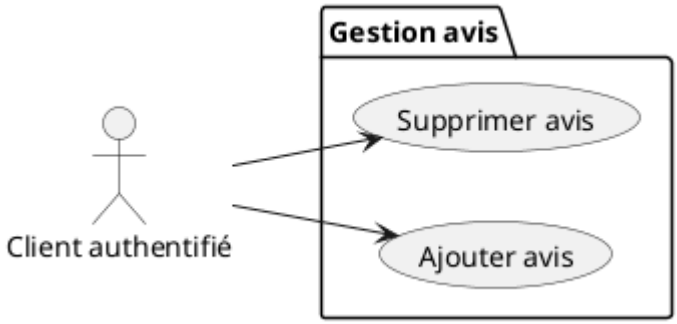
- Rechercher des annonces

- Trier et filtrer les résultats
- Consulter les détails d'une annonce



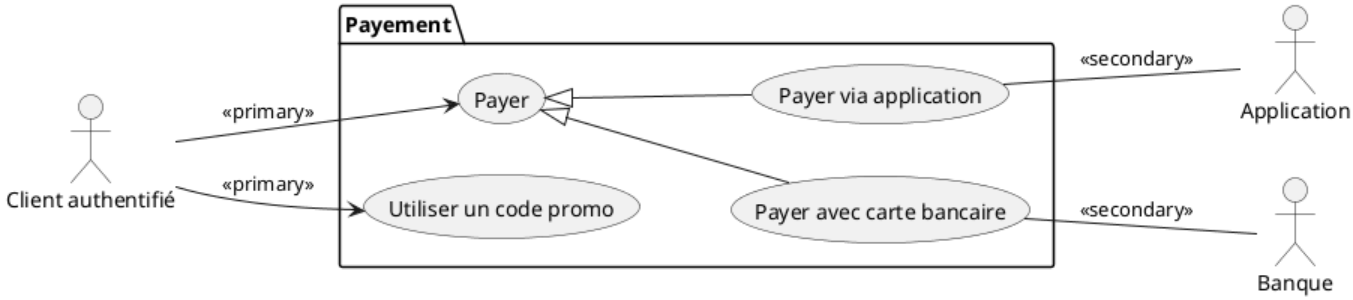
Gestion des avis

- Consulter les avis d'un vendeur
- Laisser un avis
- Supprimer un avis



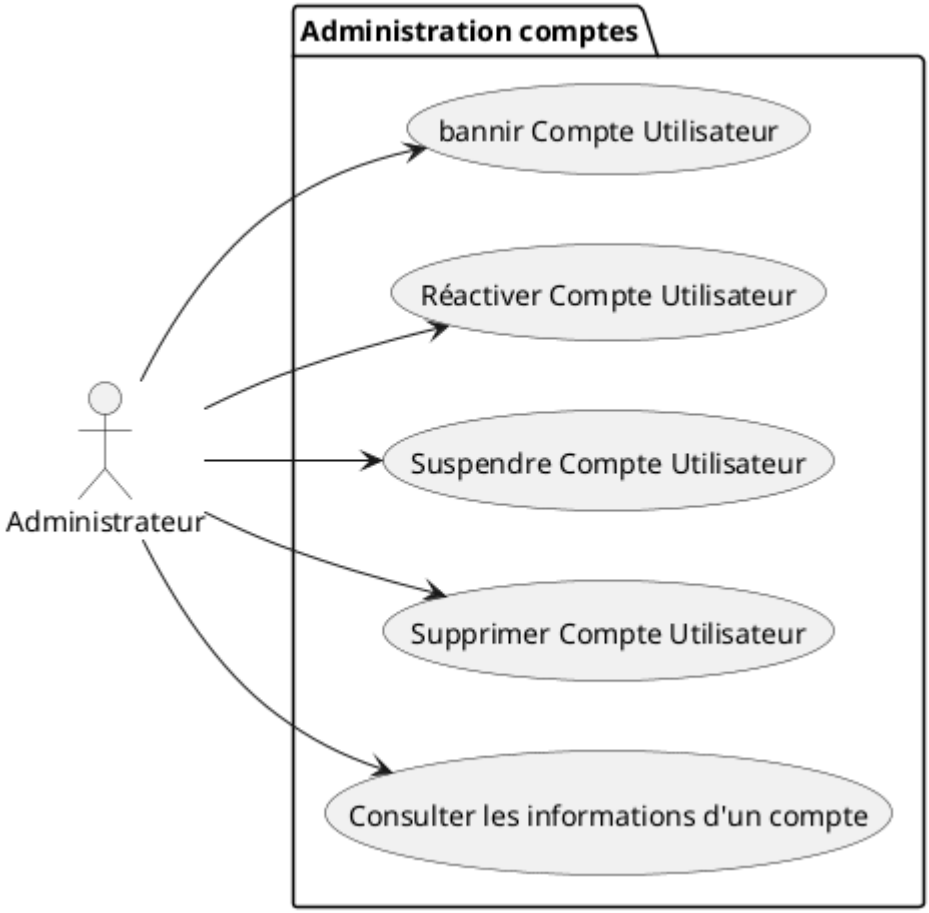
Paiement

- Procéder au paiement
- Appliquer un code promo
- Recevoir une confirmation



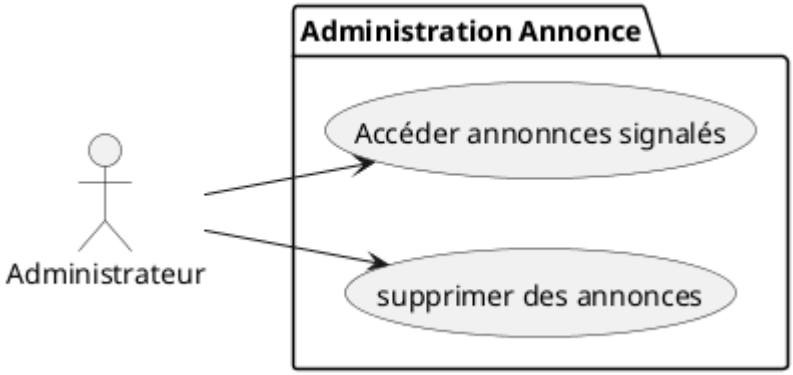
Administration des comptes

- Consulter les comptes utilisateurs
- Suspendre un compte
- Bannir un compte
- Supprimer un compte



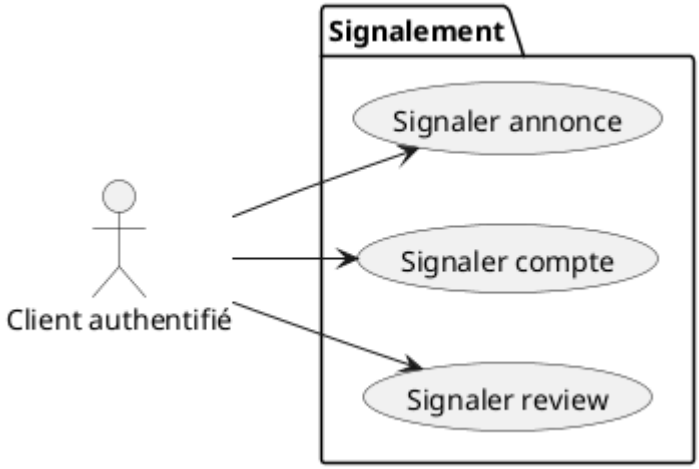
Administration des annonces

- Consulter les annonces signalées
- Supprimer une annonce



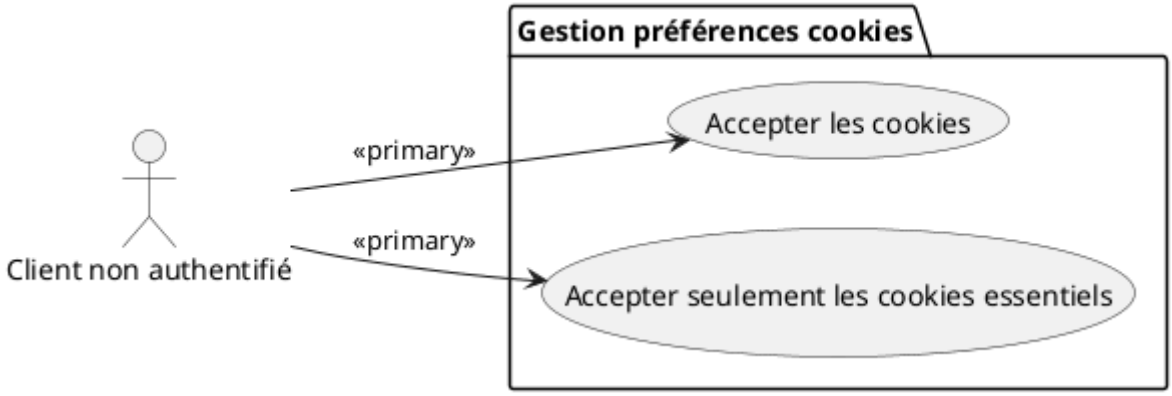
Gestion des signalements

- Signaler un compte / annonce / avis
- Traiter un signalement



Gestion des cookies

- Accepter tous les cookies
- Accepter uniquement les cookies essentiels
- Gérer les préférences de cookies

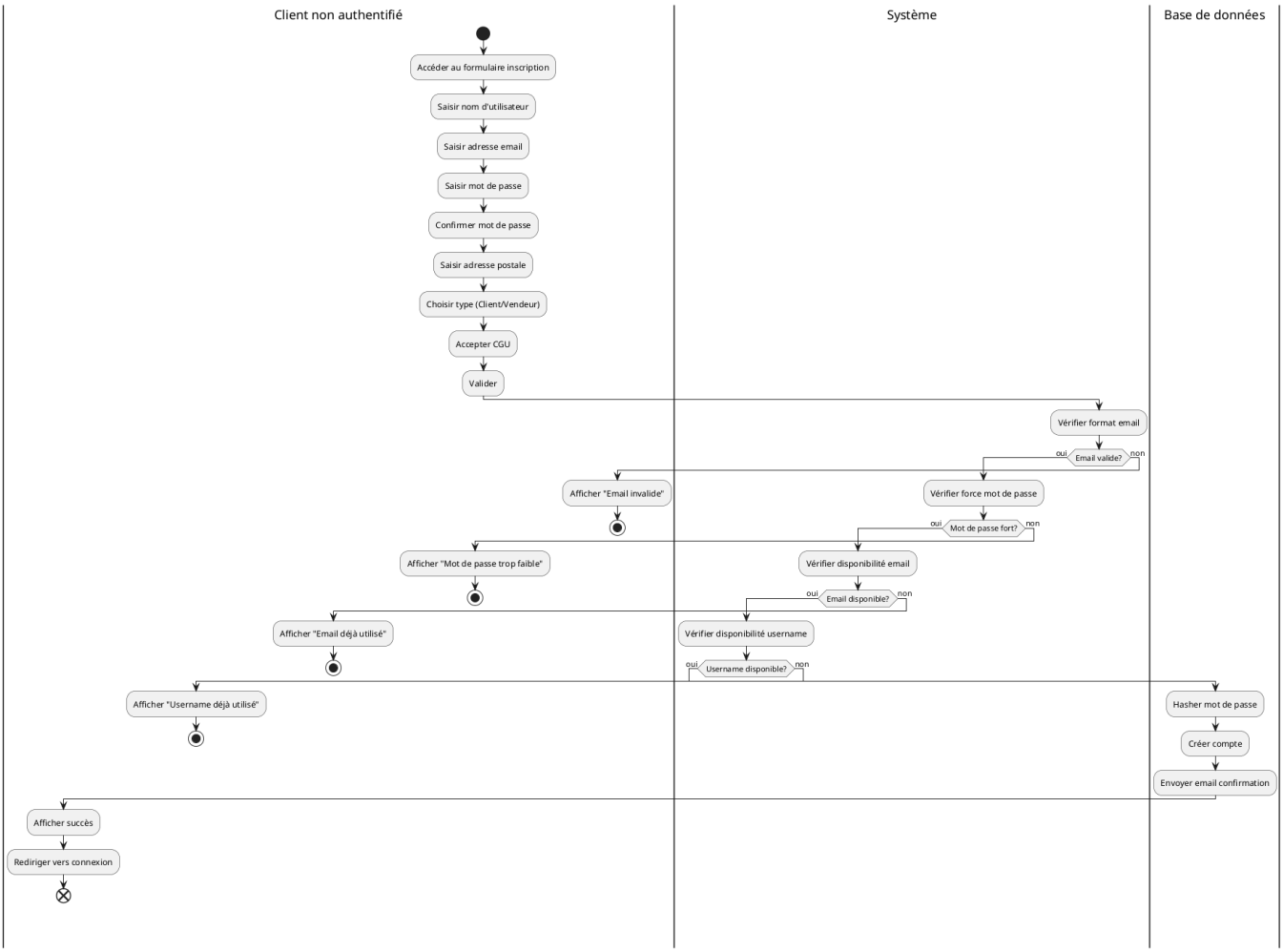


2.2 Scénarios détaillés (Diagrammes d'activités)

Les diagrammes d'activités suivants décrivent les scénarios principaux d'utilisation du système, en détaillant les flux d'actions et les décisions.

2.2.1 Gestion des comptes

Créer un compte



Description : Un visiteur crée un compte sur OnlyShoes en fournissant ses informations personnelles.

Préconditions : Aucune

Postconditions : L'utilisateur possède un compte actif et est connecté

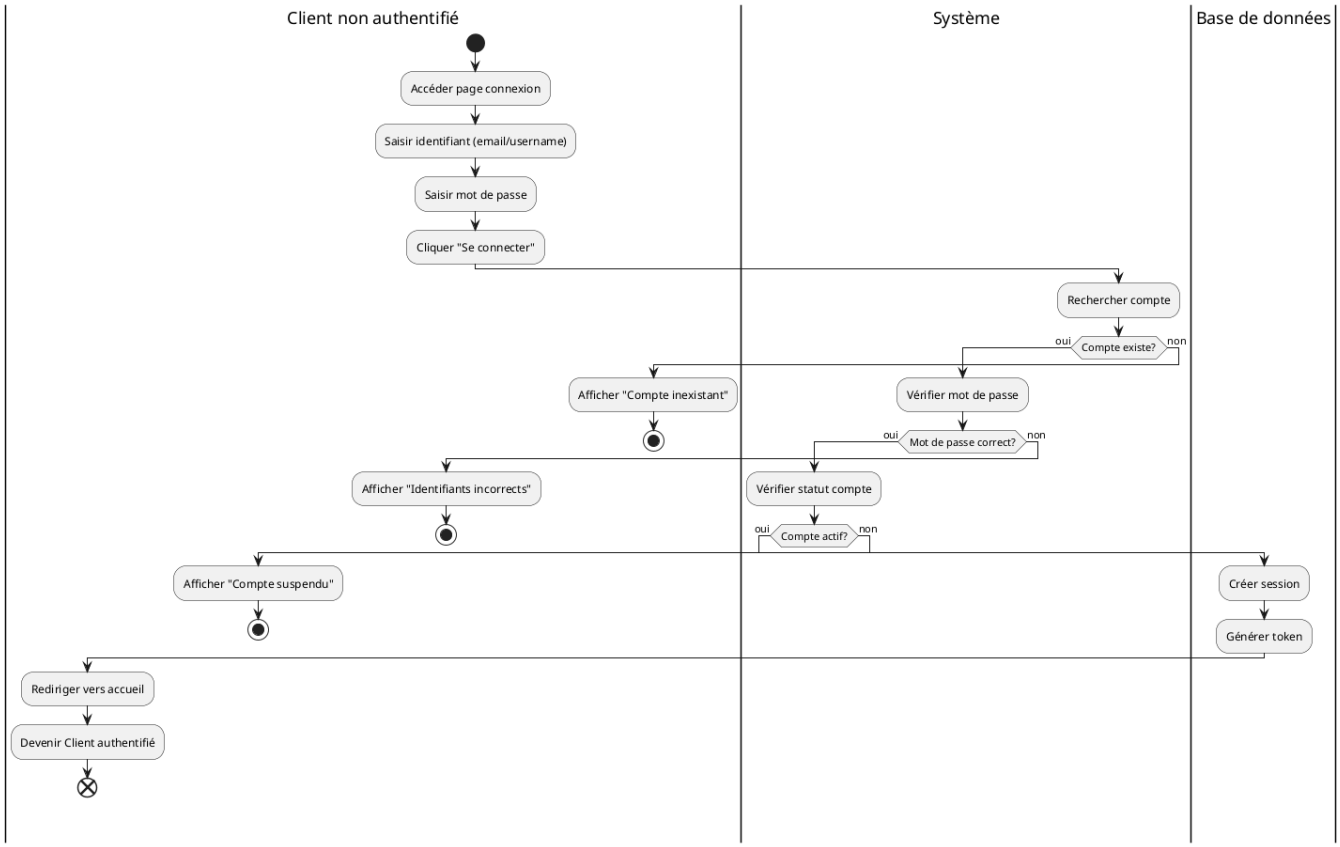
Flux principal :

1. L'utilisateur accède au formulaire d'inscription
2. Il remplit les informations (nom, prénom, email, mot de passe)
3. Le système valide les données
4. Le compte est créé
5. L'utilisateur est automatiquement connecté

Flux alternatifs :

- Email déjà utilisé : message d'erreur
- Mot de passe trop faible : demande de saisie d'un mot de passe plus fort

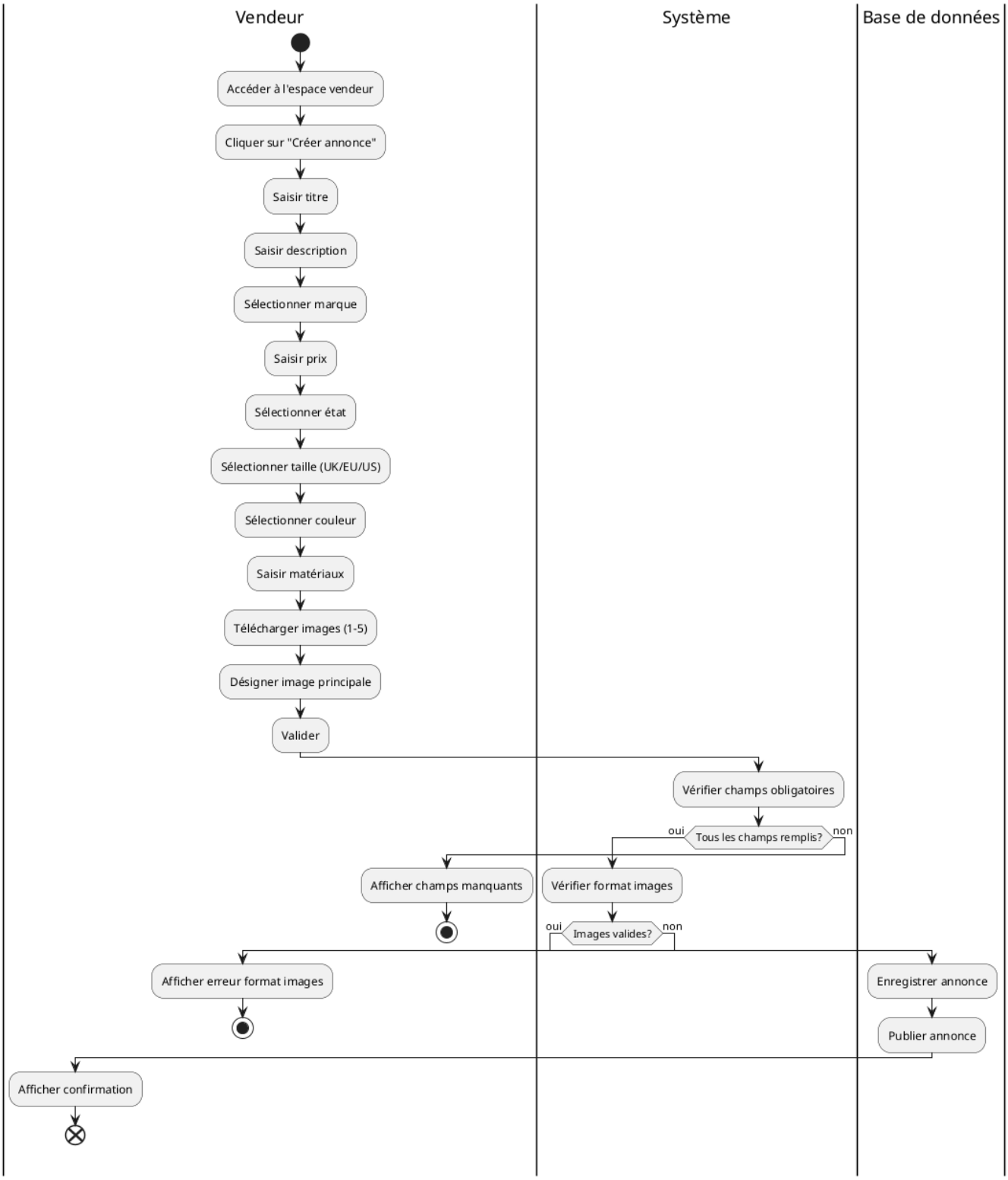
Se connecter



Description : Un utilisateur enregistré se connecte à son compte.

2.2.2 Gestion des annonces

Ajouter une annonce



Description : Un vendeur publie une nouvelle annonce de chaussures.

Préconditions : L'utilisateur doit être connecté

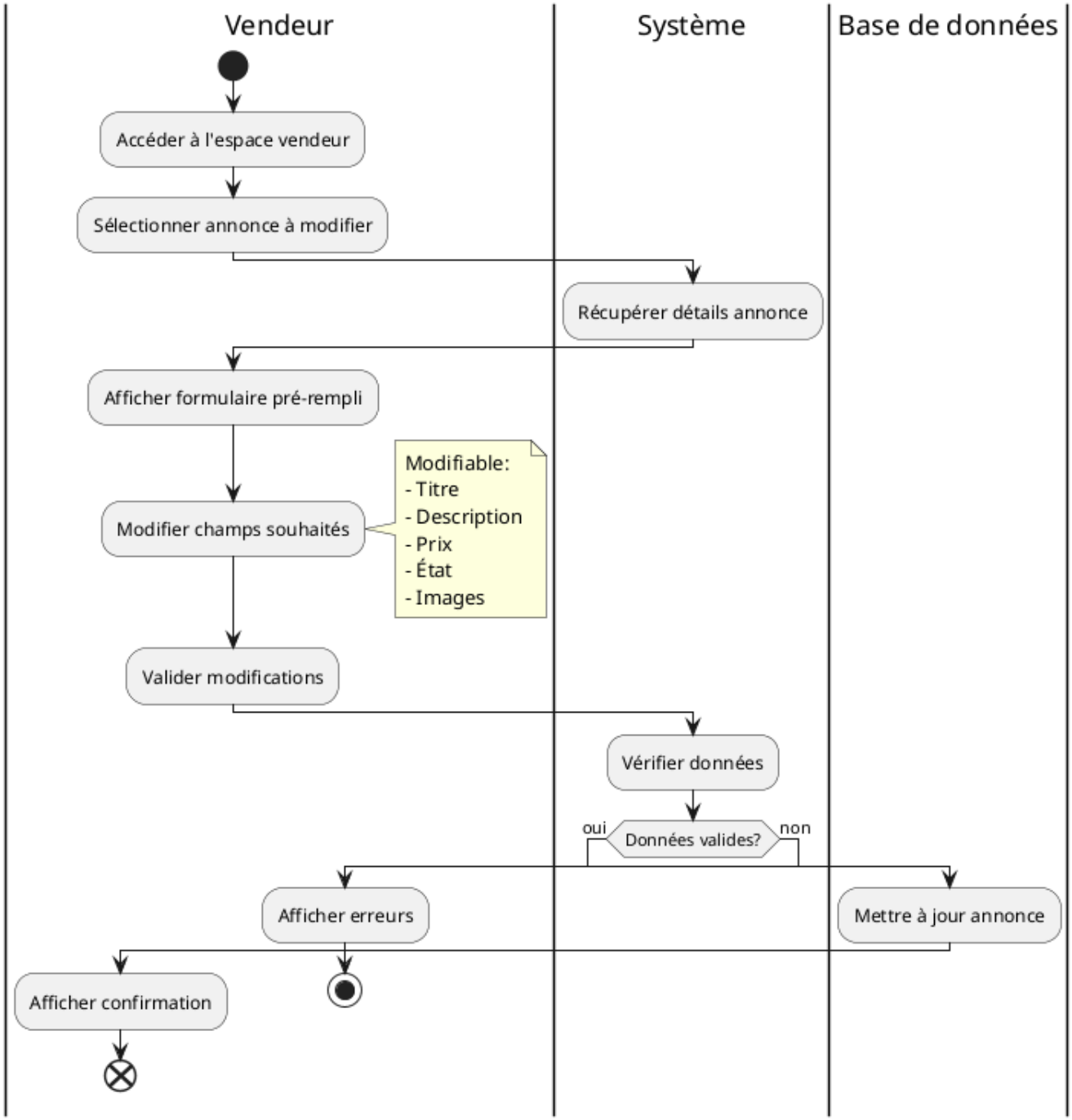
Postconditions : Une nouvelle annonce est disponible sur le site

Flux principal :

1. Le vendeur accède au formulaire de création d'annonce
2. Il remplit les informations produit (titre, description, prix, état, taille, etc.)
3. Il sélectionne la marque, couleur, matériau

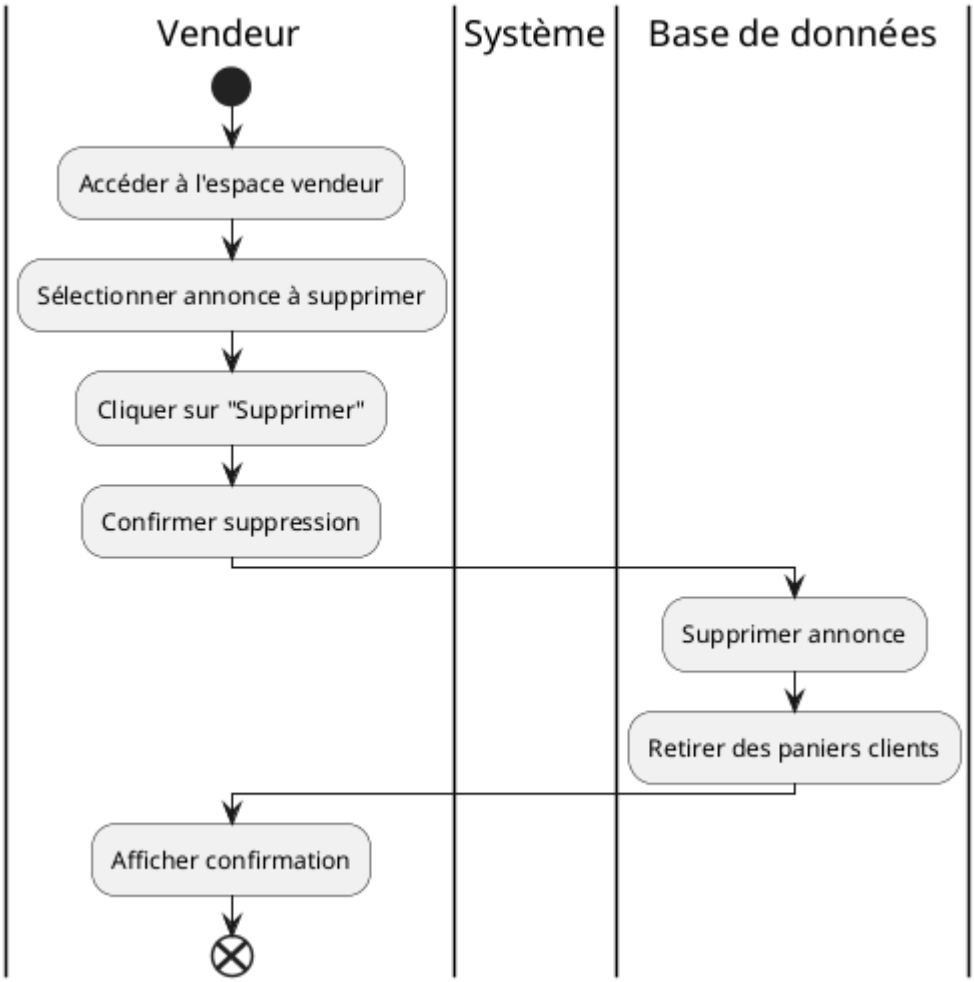
4. Il ajoute de 1 à 5 images (dont une principale)
5. Le système génère les embeddings pour la recherche sémantique
6. L'annonce est publiée

Modifier une annonce



Description : Un vendeur modifie une de ses annonces existantes.

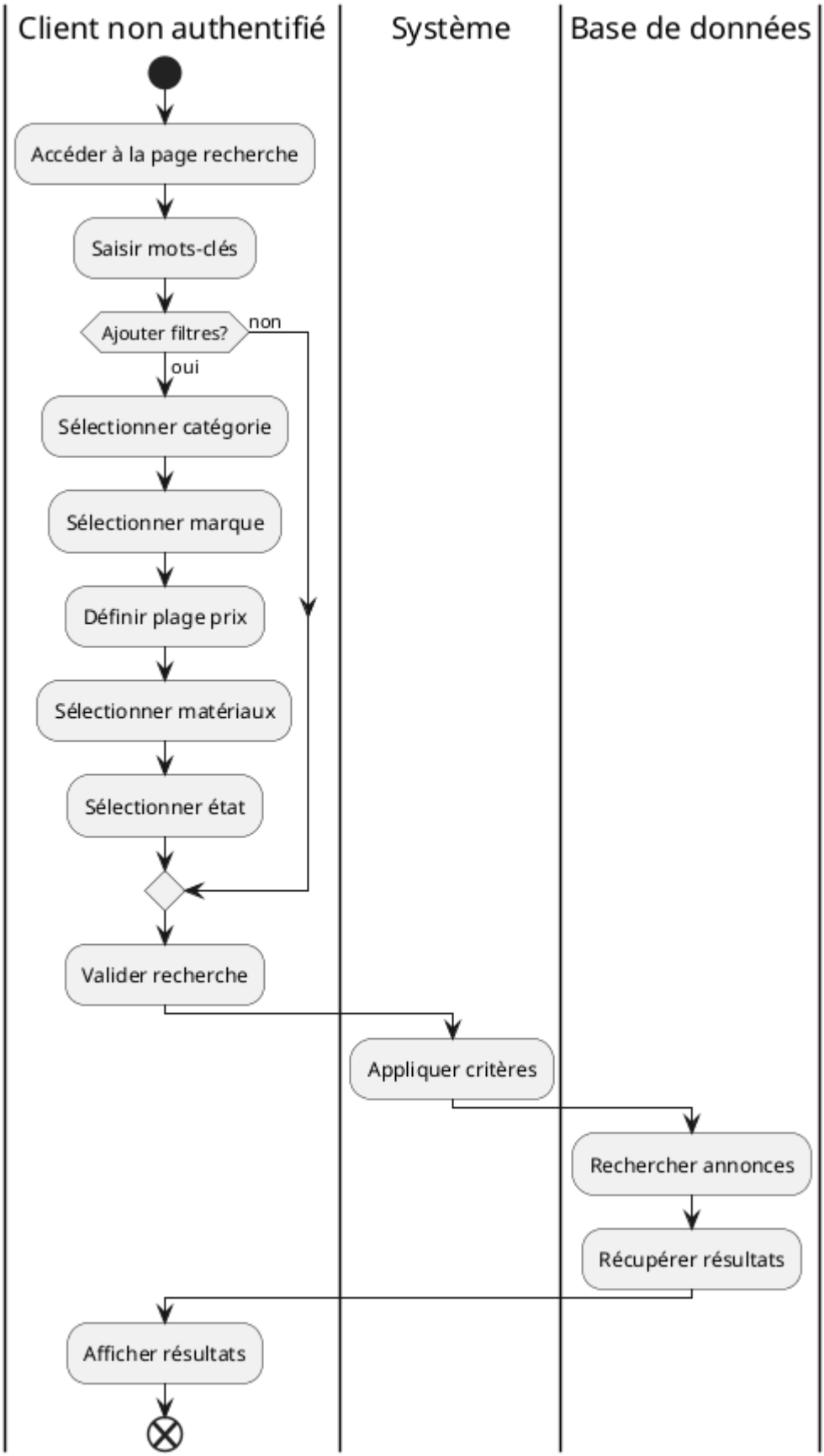
Supprimer une annonce



Description : Un vendeur supprime une de ses annonces.

2.2.3 Recherche

Rechercher des annonces



Description : Un utilisateur recherche des chaussures en utilisant des filtres.

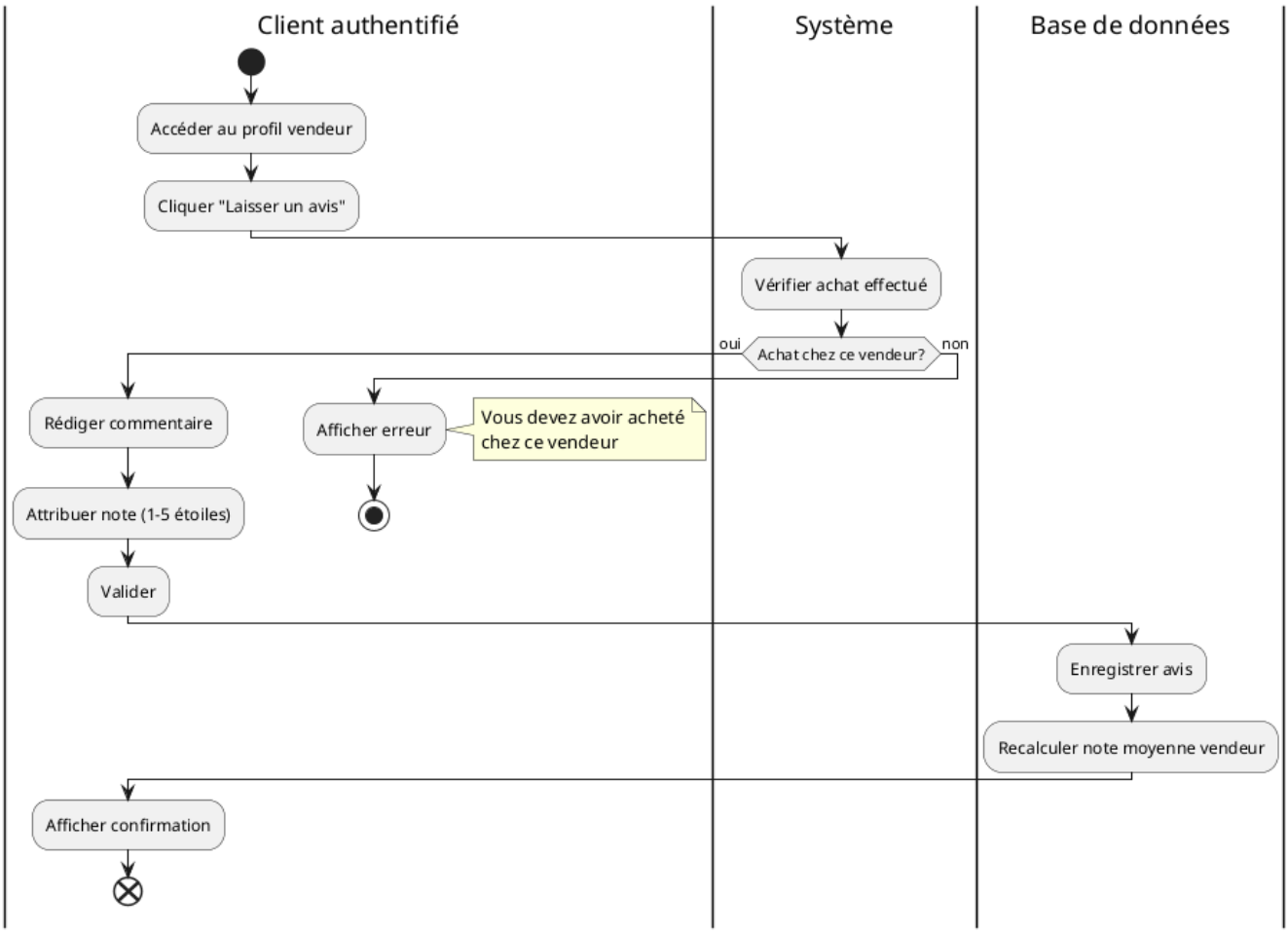
Fonctionnalités de recherche :

- Recherche textuelle (titre, description, marque)
- Recherche sémantique (embeddings)

- Filtres : prix, état, taille, couleur, matériau
- Tri : prix croissant/décroissant, popularité, nouveauté

2.2.4 Gestion des avis

Ajouter un avis



Description : Un acheteur laisse un avis sur un vendeur.

Préconditions :

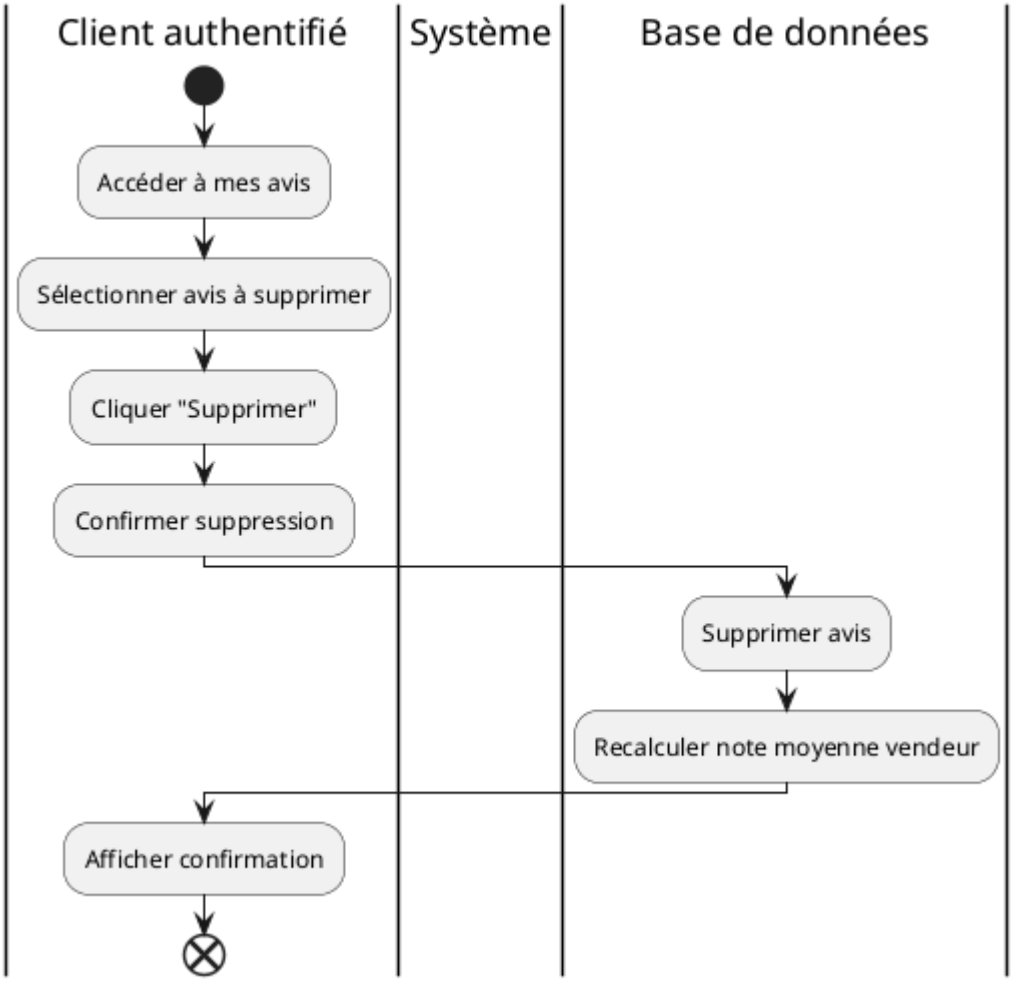
- L'utilisateur doit être connecté
- Il doit avoir effectué un achat auprès du vendeur
- L'avis porte sur le vendeur, pas sur l'article

Postconditions : L'avis est visible sur le profil du vendeur

Flux principal :

1. L'acheteur accède à la page du vendeur
2. Il clique sur "Laisser un avis"
3. Il attribue une note (1 à 5 étoiles)
4. Il rédige un commentaire (optionnel)
5. Le système vérifie qu'un achat a été effectué
6. L'avis est publié

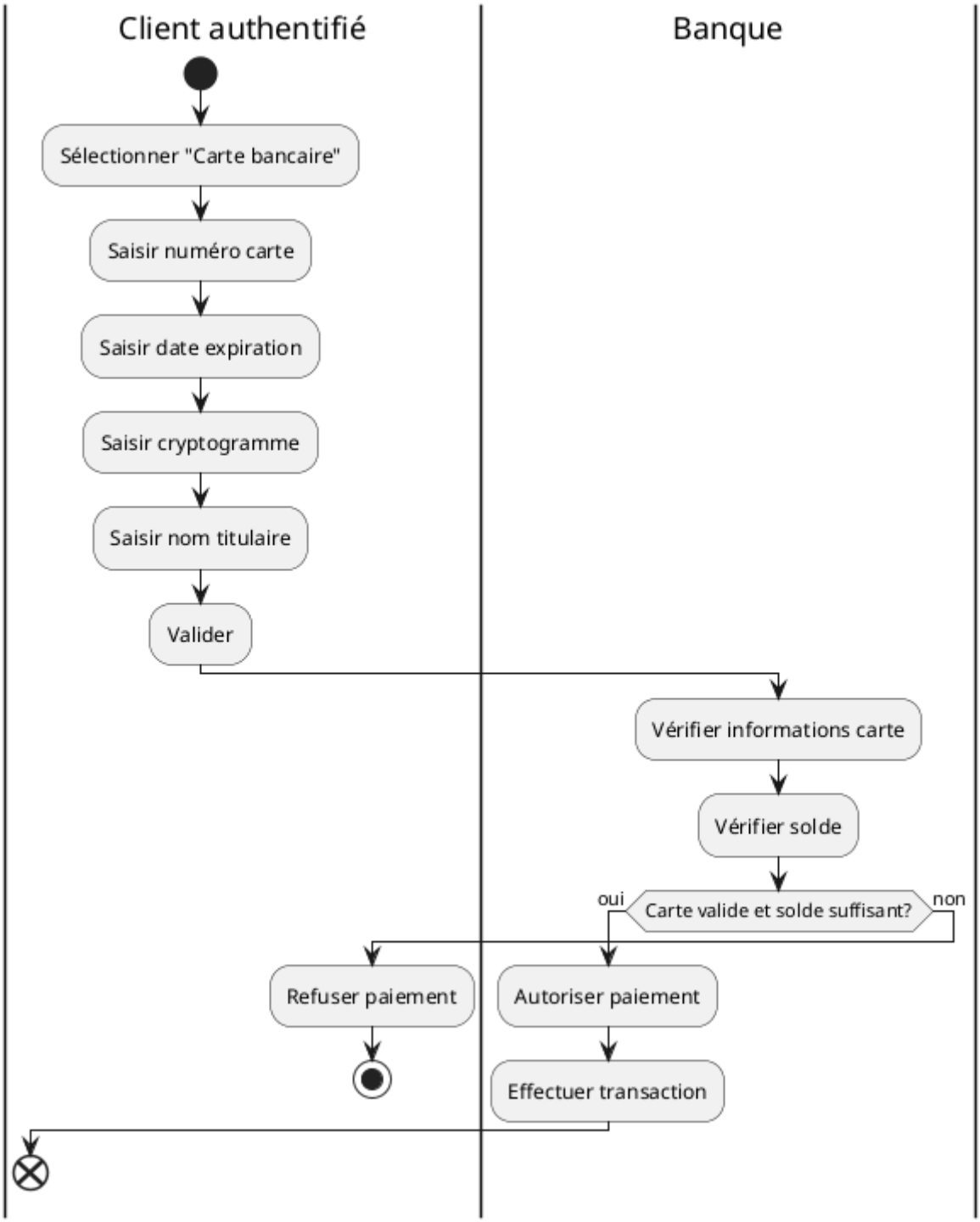
Supprimer un avis



Description : Un utilisateur ou un administrateur supprime un avis.

2.2.5 Paiement

Payer avec carte bancaire



Description : Un acheteur procède au paiement sécurisé de son achat.

Préconditions : L'utilisateur a sélectionné un article

Postconditions : Une commande est créée et le paiement est traité

Flux principal :

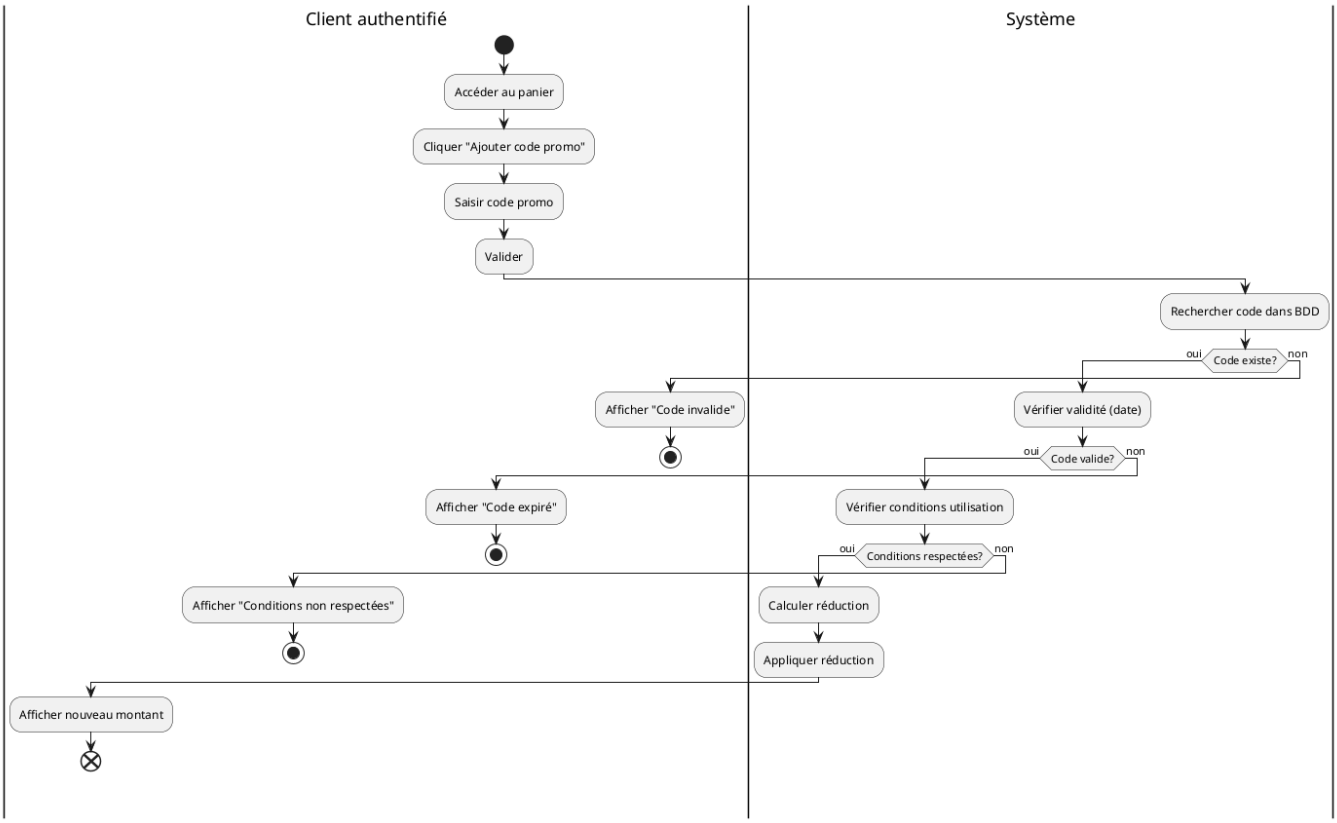
1. L'utilisateur accède au panier
2. Il applique un code promo (optionnel)
3. Il valide le panier
4. Il choisit le mode de paiement (carte bancaire, PayPal, etc.)
5. Il est redirigé vers le service de paiement sécurisé

6. Le paiement est validé
7. Une confirmation est envoyée par email au client et au vendeur
8. L'annonce est marquée comme vendue

Flux alternatifs :

- Paiement refusé : l'utilisateur est informé et peut réessayer
- Code promo invalide : message d'erreur

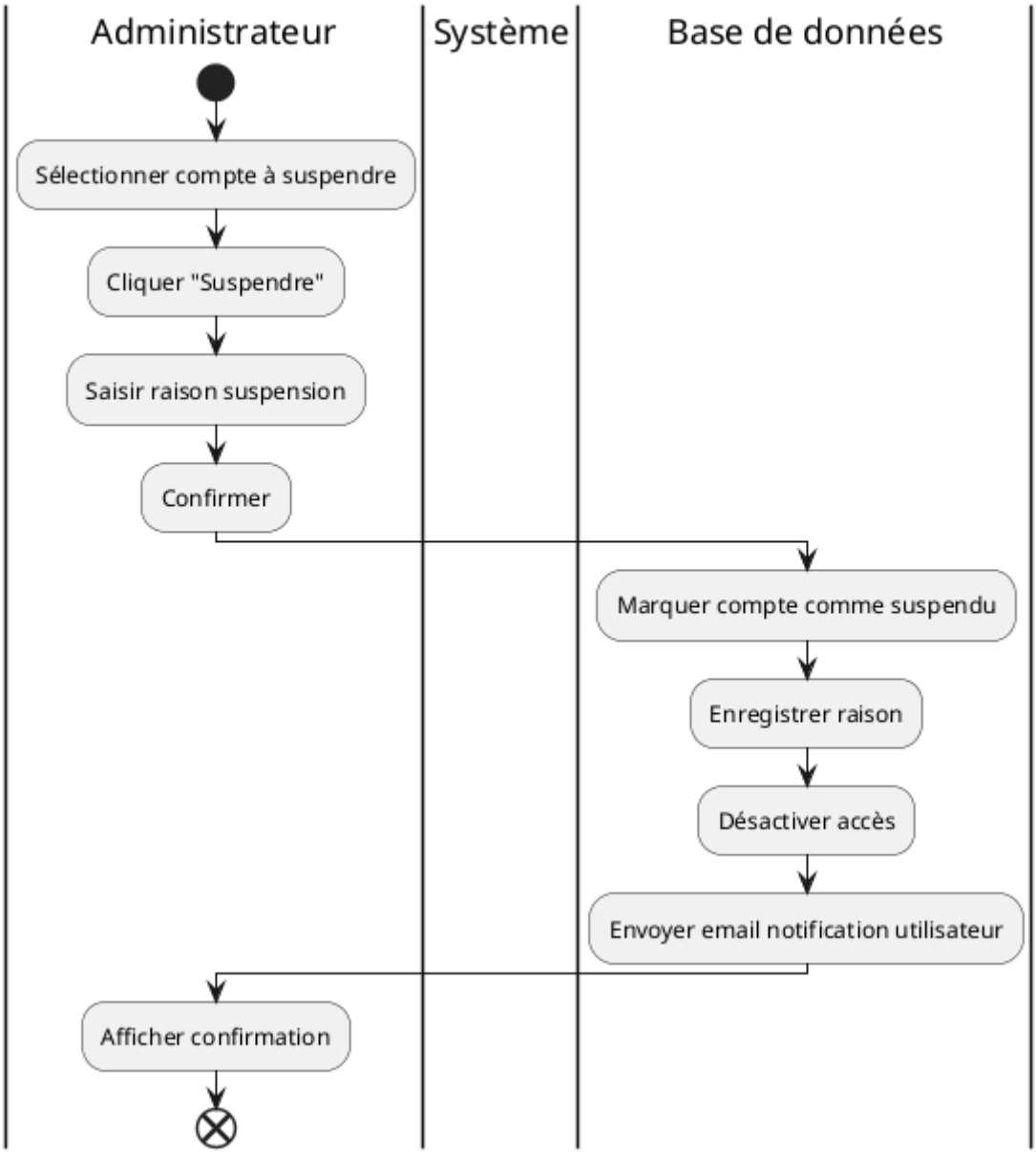
Utiliser un code promo



Description : Application d'un code de réduction avant paiement.

2.2.6 Administration

Suspendre un compte utilisateur



Description : Un administrateur suspend temporairement un compte utilisateur.

Préconditions : L'administrateur doit être connecté avec les droits appropriés

Postconditions : Le compte est suspendu et l'action est loggée

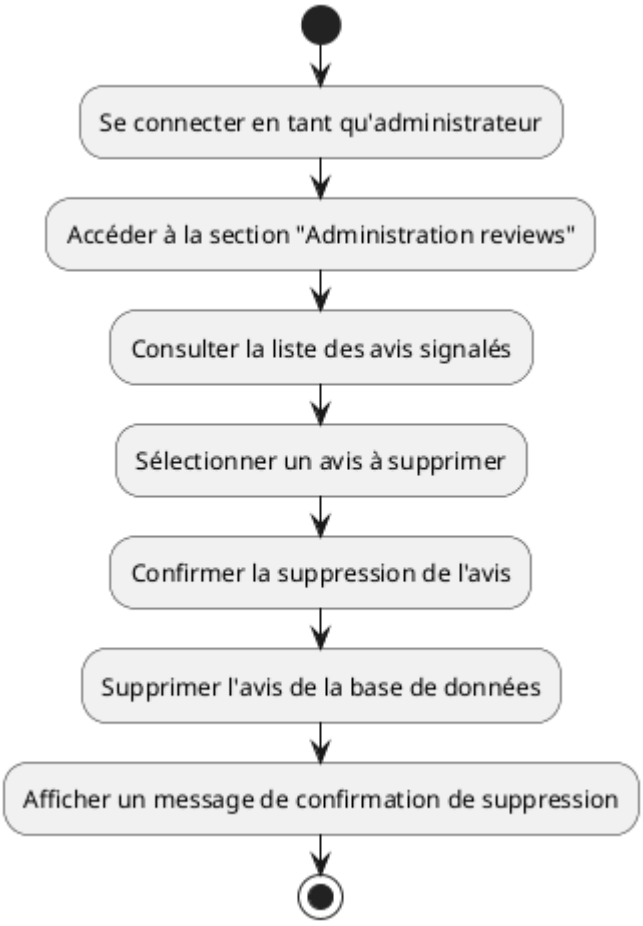
Flux principal :

1. L'administrateur accède à la liste des utilisateurs
2. Il sélectionne l'utilisateur à suspendre
3. Il renseigne une raison
4. Le compte passe au statut "suspendu"
5. L'action est enregistrée dans les logs administrateur
6. L'utilisateur est notifié

Consulter les avis signalés

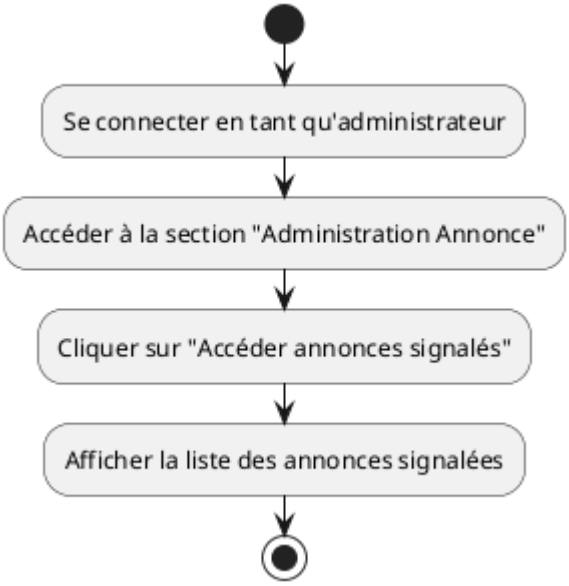
Description : Un administrateur consulte les avis qui ont été signalés par les utilisateurs.

Supprimer un avis



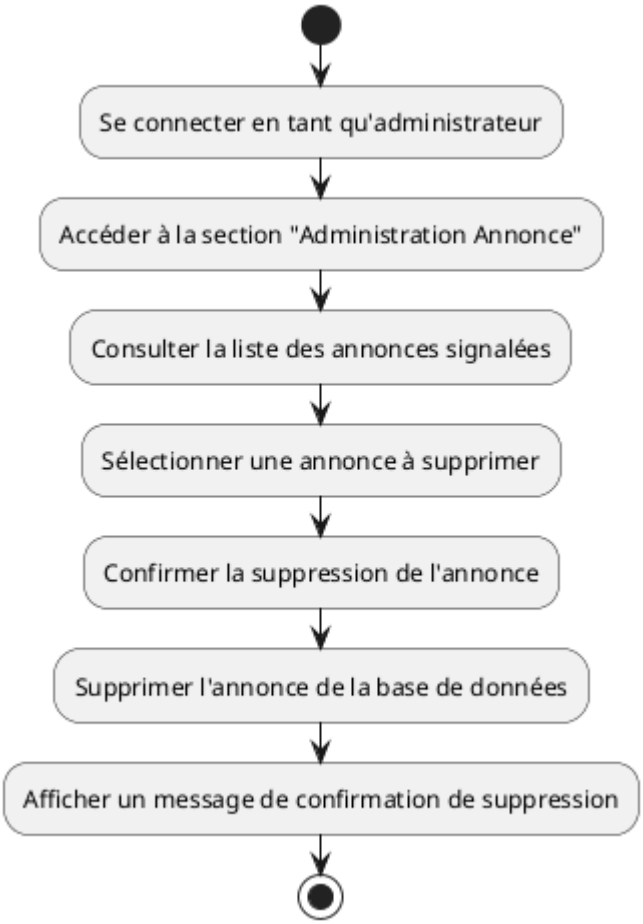
Description : Un administrateur supprime un avis inapproprié.

Accéder aux annonces signalées



Description : Un administrateur consulte les annonces qui ont été signalées.

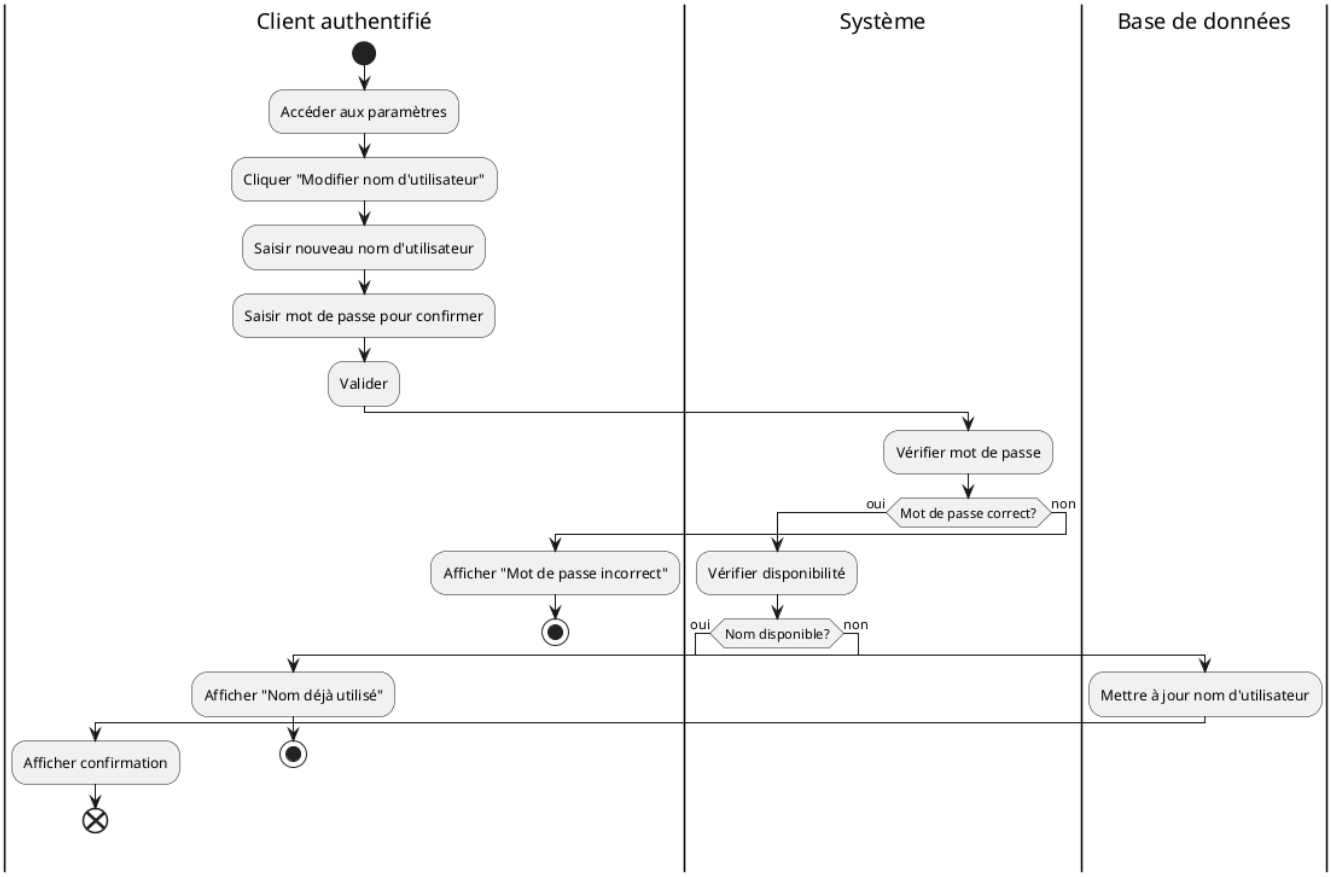
Supprimer une annonce (admin)



Description : Un administrateur supprime une annonce signalée comme inappropriée.

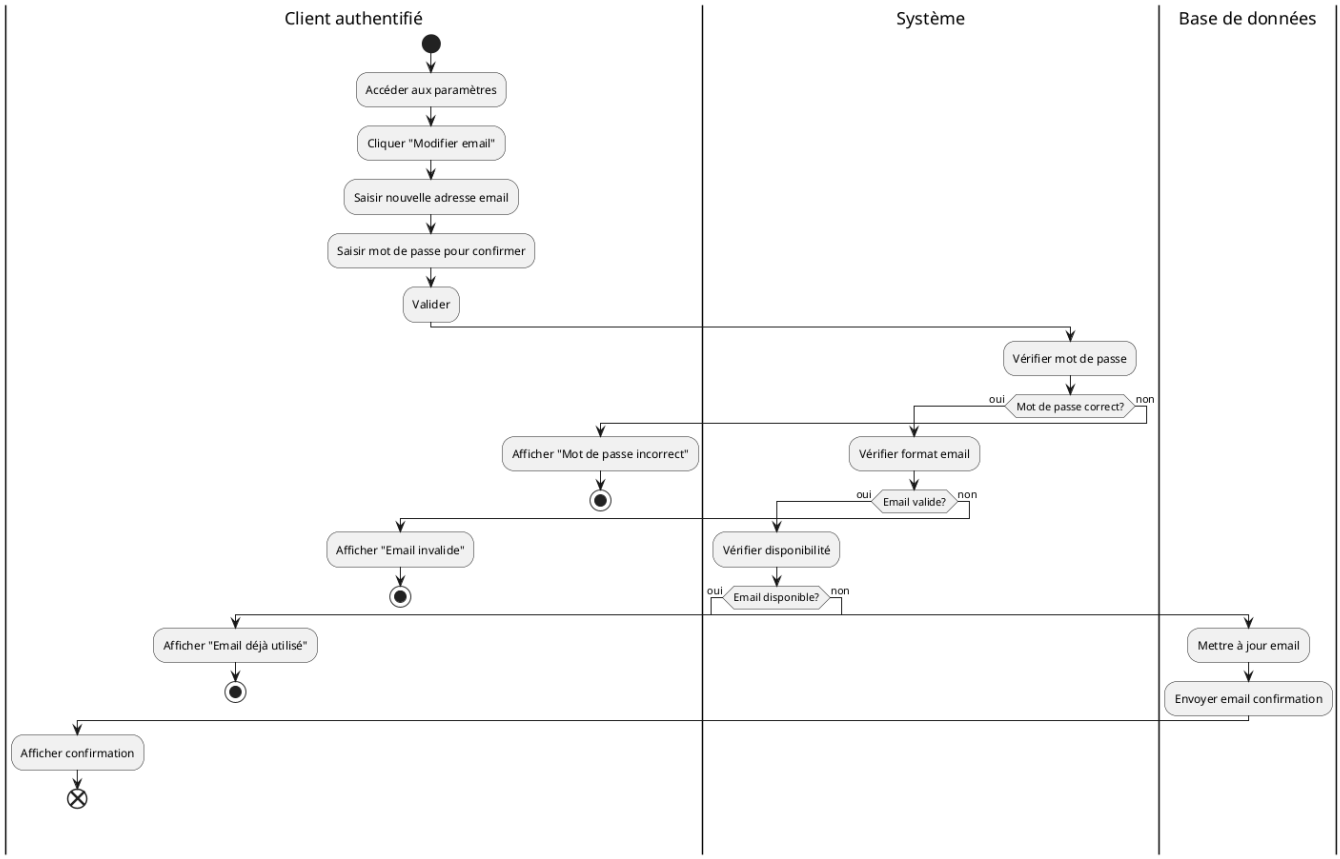
2.2.7 Modifications des informations

Modifier le nom d'utilisateur



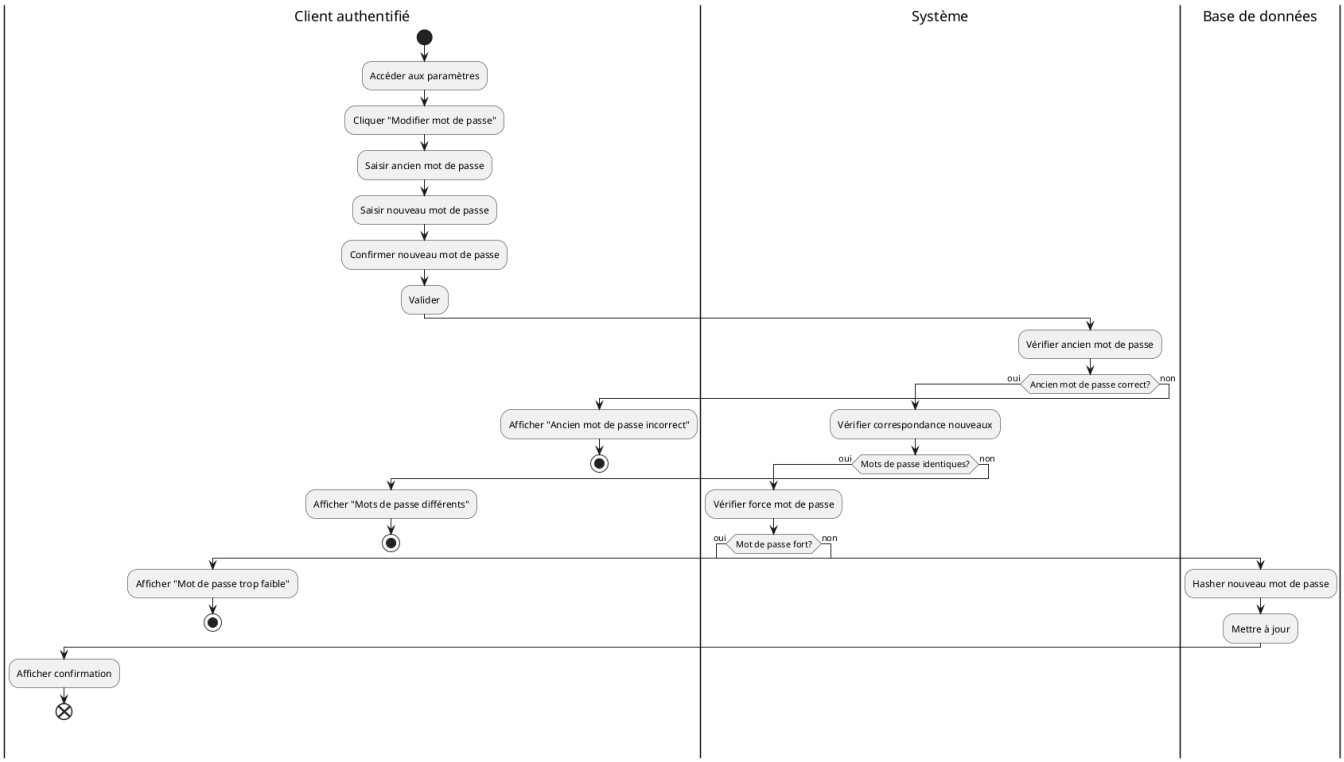
Description : Un utilisateur modifie son nom d'utilisateur.

Modifier l'adresse email



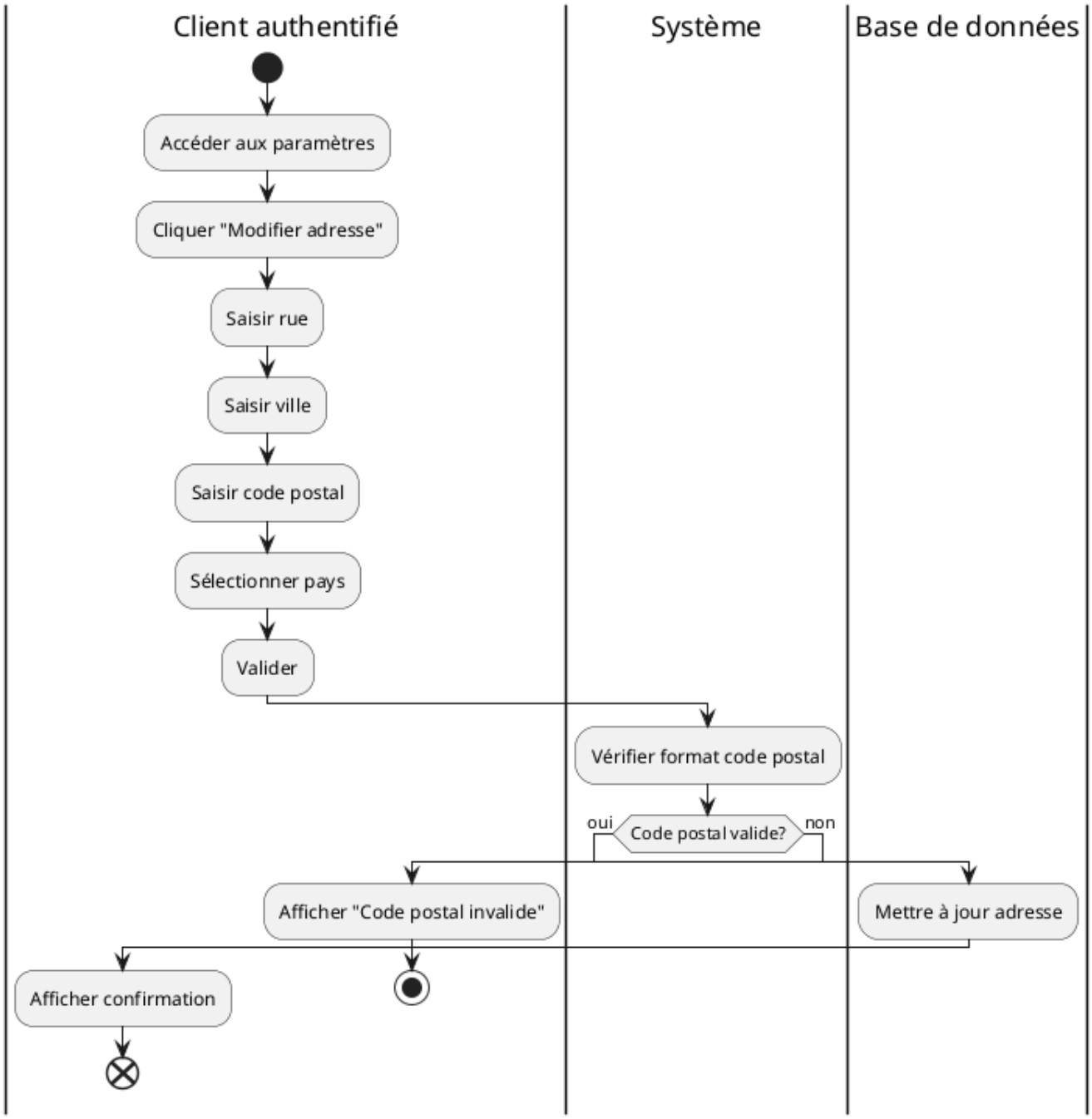
Description : Un utilisateur change son adresse email.

Modifier le mot de passe



Description : Un utilisateur change son mot de passe.

Modifier l'adresse postale



Description : Un utilisateur met à jour son adresse de livraison.

---

3. Diagramme de Classes d'Analyse

3.1 Vue d'ensemble complète

Le diagramme de classes d'analyse suivant présente le modèle métier complet du système OnlyShoes, avec toutes les entités identifiées et leurs associations.

Diagramme de classes d'analyse complet

Entités principales identifiées

Le système OnlyShoes repose sur 14 entités métier organisées en 5 domaines fonctionnels :

Domaine Utilisateur

- **Utilisateur** : Entité centrale représentant un compte utilisateur (client, vendeur ou administrateur)
- **Adresse** : Adresse postale associée à un utilisateur
- **ConsentementUtilisateur** : Gestion des consentements RGPD (cookies, traitement données, etc.)
- **AdminLog** : Historique des actions administratives

Domaine Annonce

- **Annonce** : Produit (paire de chaussures) mis en vente
- **Image** : Photo d'une annonce (1 à 5 images par annonce)
- **Marque** : Marque de chaussures (Nike, Adidas, etc.)
- **Couleur** : Couleur du produit
- **Materiau** : Matériau de composition (cuir, toile, etc.)

Domaine Commande

- **Commande** : Achat effectué par un client
- **LigneCommande** : Ligne détaillant un article dans une commande
- **Paiement** : Transaction bancaire associée à une commande

Domaine Avis

- **Review** : Avis laissé par un acheteur sur un vendeur

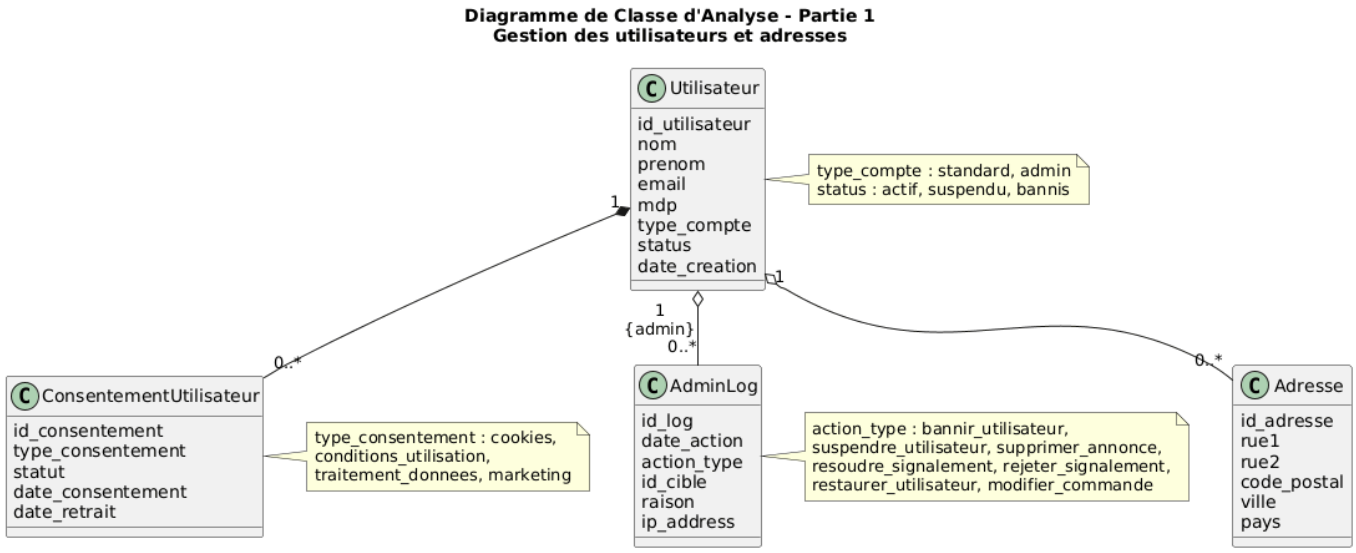
Domaine Modération

- **Signalement** : Signalement d'un contenu inapproprié (utilisateur, annonce ou avis)

3.2 Vues détaillées par domaine

Pour faciliter la lecture, le diagramme de classes d'analyse a été segmenté en 5 parties correspondant aux domaines fonctionnels.

3.2.1 Gestion des utilisateurs



Entités du domaine :

- **Utilisateur** : Compte utilisateur avec type (standard/admin) et statut (actif/suspendu/bannis)
- **Adresse** : Adresse postale pour la livraison
- **ConsentementUtilisateur** : Gestion des 4 types de consentements RGPD
- **AdminLog** : Traçabilité des actions administratives

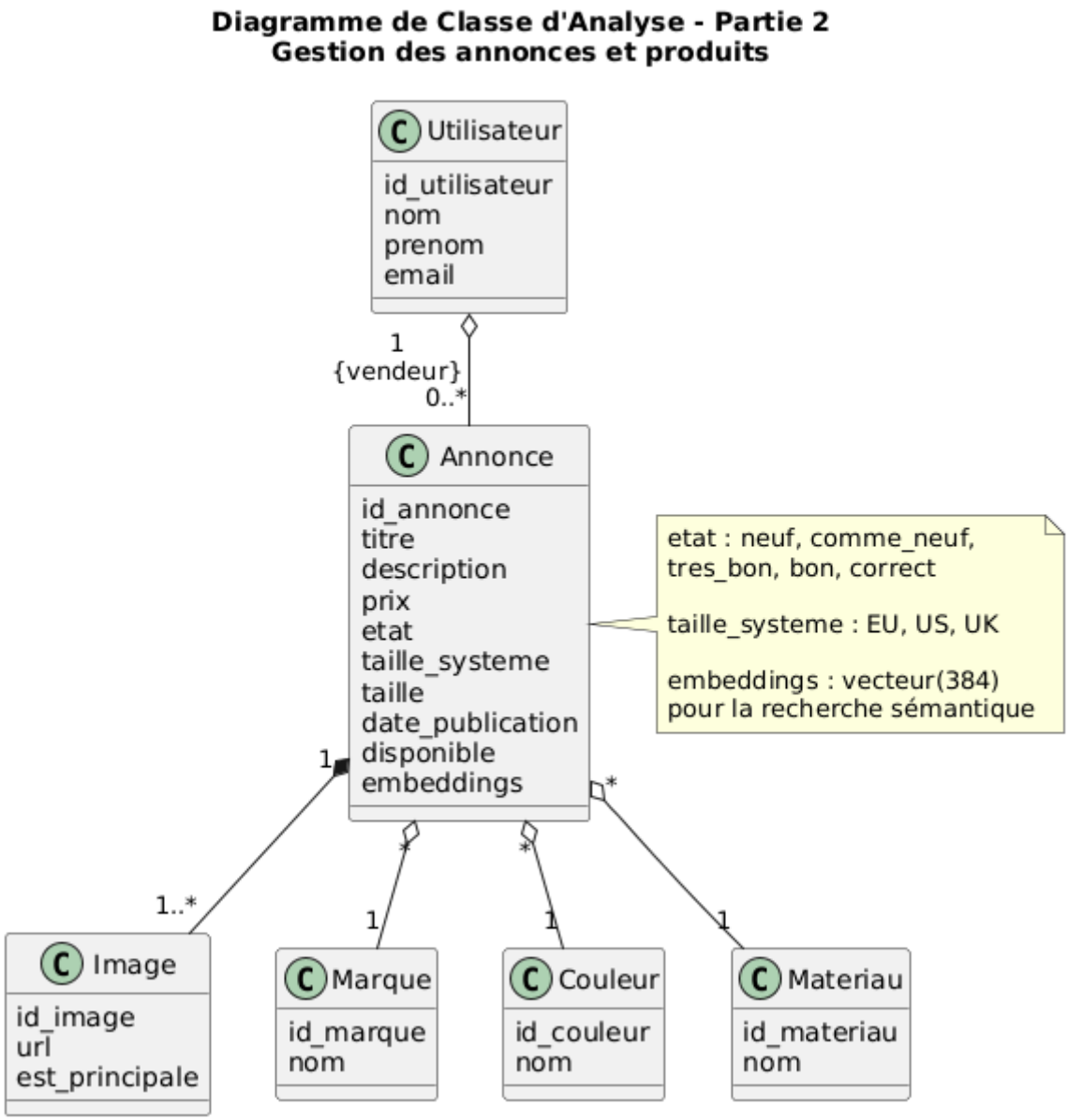
Associations :

- Un utilisateur possède 0 à plusieurs adresses
- Un utilisateur donne 0 à plusieurs consentements
- Un administrateur effectue 0 à plusieurs actions loggées

Règles métier :

- `type_compte` : standard ou admin
- `status` : actif, suspendu ou bannis
- Les types de consentement incluent : cookies, conditions_utilisation, traitement_donnees, marketing

3.2.2 Gestion des annonces et produits



Entités du domaine :

- **Annonce** : Produit mis en vente avec toutes ses caractéristiques
- **Image** : Photo du produit (jusqu'à 5 images, dont 1 principale)
- **Marque** : Référentiel des marques
- **Couleur** : Référentiel des couleurs
- **Materiau** : Référentiel des matériaux

Associations :

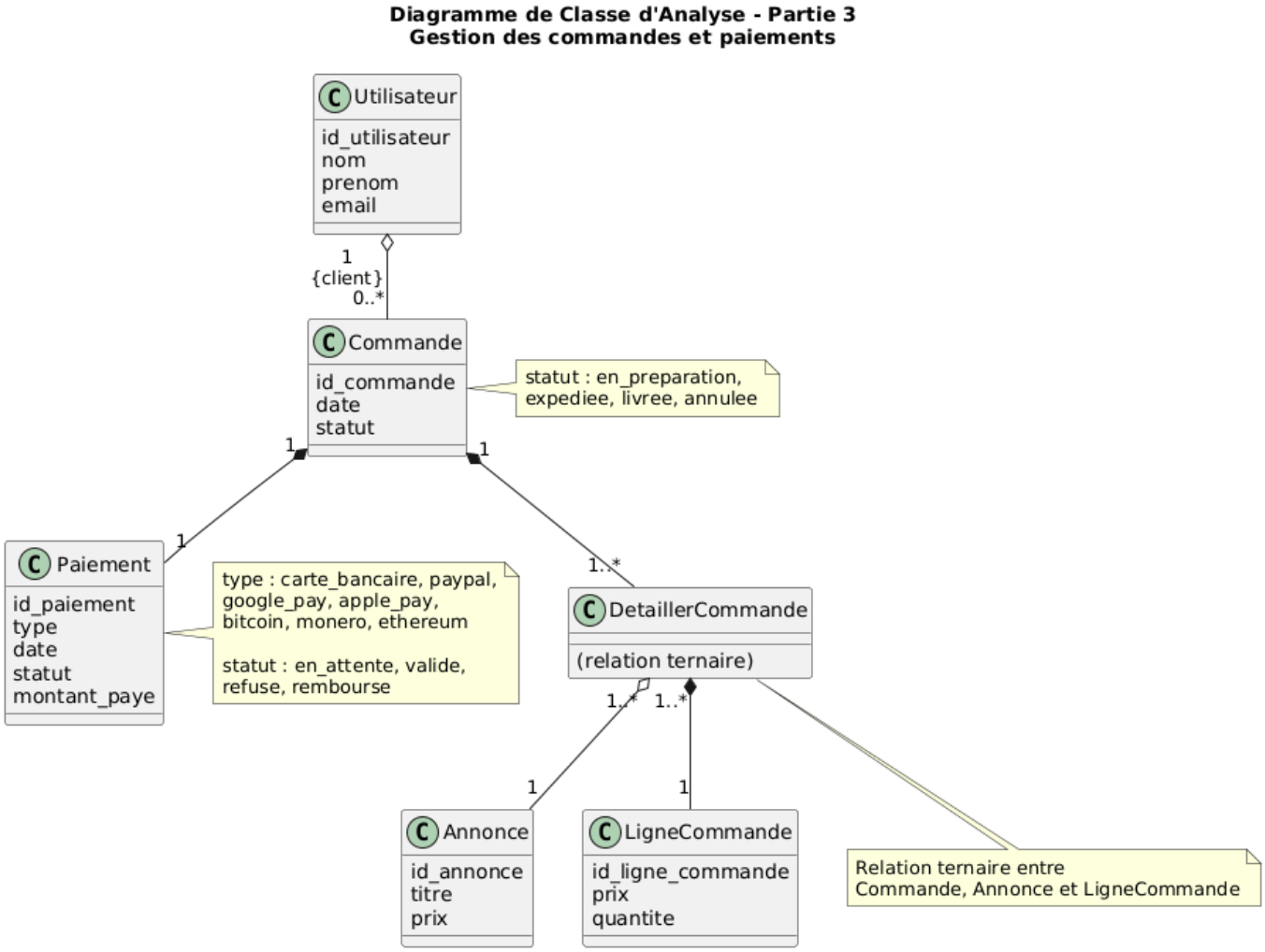
- Un vendeur publie 0 à plusieurs annonces
- Une annonce est illustrée par 1 à plusieurs images
- Une annonce appartient à 1 marque
- Une annonce a 1 couleur
- Une annonce est composée d'1 matériau

Règles métier :

- `etat` : neuf, comme_neuf, tres_bon, bon, correct
- `taille_systeme` : EU, US, UK

- `embeddings` : vecteur(384) pour la recherche sémantique

3.2.3 Gestion des commandes et paiements



Entités du domaine :

- **Commande** : Achat effectué par un client
- **LigneCommande** : Détail d'une ligne de commande (prix, quantité)
- **Paiement** : Transaction bancaire
- **DetaillerCommande** : Relation ternaire entre Commande, Annonce et LigneCommande

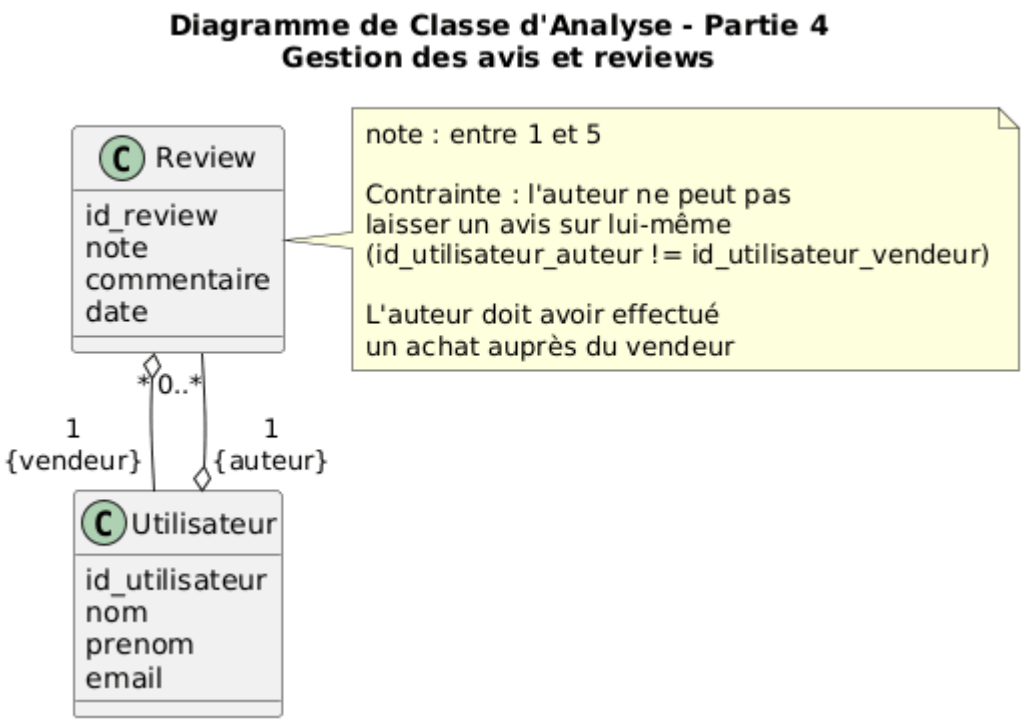
Associations :

- Un client passe 0 à plusieurs commandes
- Une commande est réglée par 1 paiement
- Une commande contient 1 à plusieurs lignes de commande
- Une ligne de commande concerne 1 annonce

Règles métier :

- `statut` commande : en_preparation, expediee, livree, annulee
- `type` paiement : carte_bancaire, paypal, google_pay, apple_pay, bitcoin, monero, ethereum
- `statut` paiement : en_attente, valide, refuse, rembourse

3.2.4 Gestion des avis



Entités du domaine :

- **Review** : Avis avec note (1-5) et commentaire optionnel

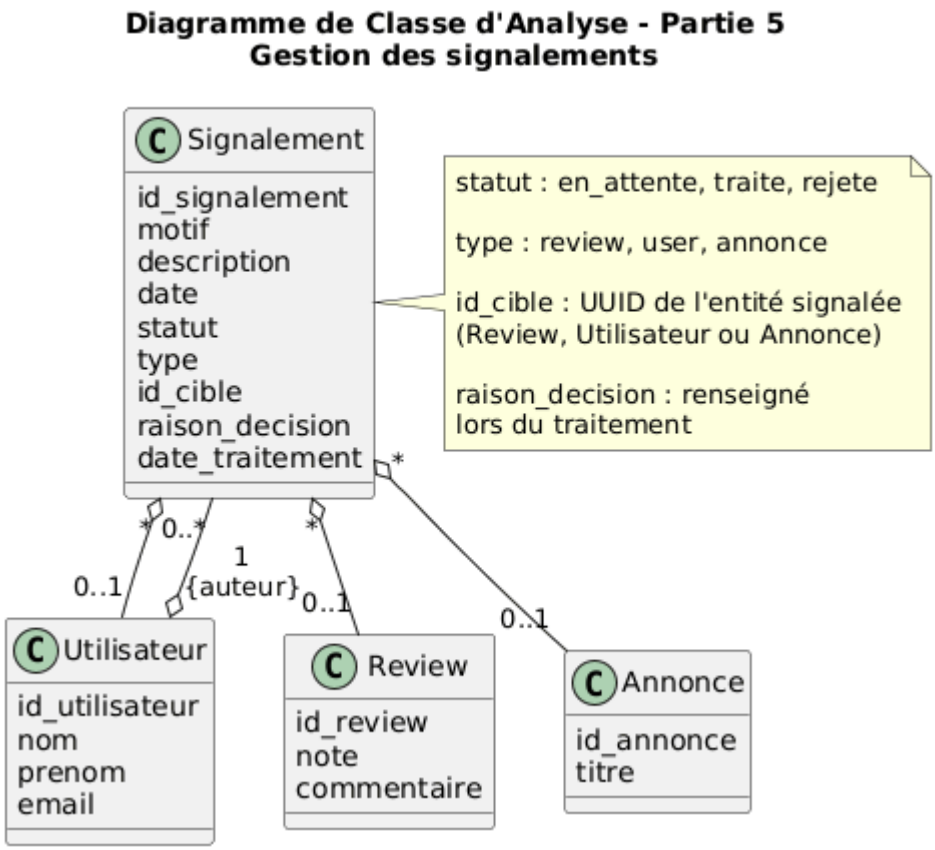
Associations :

- Un auteur écrit 0 à plusieurs reviews
- Un vendeur reçoit 0 à plusieurs reviews

Règles métier :

- La note est comprise entre 1 et 5
- L'auteur ne peut pas laisser un avis sur lui-même (id_utilisateur_auteur ≠ id_utilisateur_vendeur)
- L'auteur doit avoir effectué un achat auprès du vendeur pour laisser un avis

3.2.5 Gestion des signalements



Entités du domaine :

- **Signalement** : Déclaration d'un contenu inapproprié

Associations :

- Un auteur émet 0 à plusieurs signalements
- Un signalement peut cibler :
  - Un utilisateur (si type = user)
  - Une annonce (si type = annonce)
  - Un avis (si type = review)

Règles métier :

- `statut` : en_attente, traite, rejete
- `type` : review, user, annonce
- `id_cible` : UUID de l'entité signalée (polymorphique)
- Lors du traitement, une raison de décision doit être renseignée

---

3.3 Contraintes et règles métier

Justifications des choix de conception

Le diagramme de classes d'analyse a été conçu pour répondre spécifiquement aux exigences du cahier des charges :

1. Système de recherche avancé (Exigences 3, 4, 5)

Choix : Ajout d'un attribut `embeddings` (vecteur 384D) dans l'entité Annonce.

Justification : Le cahier des charges exige une recherche par critères multiples. Les embeddings permettent une recherche sémantique intelligente qui va au-delà de la simple correspondance de mots-clés, améliorant significativement l'expérience utilisateur.

2. Gestion granulaire des consentements (Exigences 10, 11)

Choix : Entité `ConsentementUtilisateur` avec 4 types de consentement distincts.

Justification : Le RGPD impose une granularité des consentements. L'utilisateur peut choisir indépendamment d'accepter les cookies essentiels, les cookies fonctionnels, le traitement de ses données, et les communications marketing.

3. Système de modération (Exigence 35)

Choix : Entité `Signalement` polymorphique pouvant cibler un utilisateur, une annonce ou un avis.

Justification : L'exigence 35 demande la possibilité de signaler divers types de contenus. Une entité unique avec un type discriminant simplifie la gestion tout en permettant la traçabilité requise.

4. Traçabilité administrative (Exigences 31-34)

Choix : Entité `AdminLog` enregistrant toutes les actions administratives.

Justification : Les actions de modération (suspension, bannissement, suppression) doivent être tracées pour des raisons légales et de sécurité. Chaque action inclut la raison, l'IP et l'horodatage.

5. Système de notation des vendeurs (Exigences 18, 19)

Choix : Entité `Review` liée au vendeur et non à l'annonce.

Justification : Le cahier des charges stipule que les avis concernent les vendeurs pour établir une réputation. Un vendeur avec de bonnes notes inspire confiance, encourageant les transactions.

6. Paiement sécurisé externalisé (Exigence 25)

Choix : Entité `Paiement` avec type et référence externe.

Justification : Pour des raisons de sécurité (certification PCI-DSS), les paiements sont délégués à des services tiers (Stripe, PayPal). Seules les métadonnées (type, statut, référence) sont stockées localement.

7. Confirmation automatique (Exigence 26)

Choix : Triggers PostgreSQL pour l'envoi automatique d'emails.

Justification : L'exigence impose des confirmations automatiques. Les triggers garantissent l'atomicité : une commande validée déclenche systématiquement l'envoi d'un email au client et au vendeur.

8. Référentiels de données (Exigences 1, 2)

Choix : Entités `Marque`, `Couleur`, `Materiau` séparées.

Justification : Les filtres de recherche exigent des catégorisations précises. Des tables de référence normalisées facilitent les requêtes et garantissent la cohérence des données.

Contraintes d'intégrité

1. **Utilisateur**
    - L'email doit être unique
    - Le mot de passe doit être haché (bcrypt)
    - Le type de compte ne peut être que 'standard' ou 'admin'
    - Le statut ne peut être que 'actif', 'suspendu' ou 'bannis'
2. **Annonce**
    - Le prix doit être positif
    - Une annonce doit avoir au moins 1 image
    - Les embeddings sont générés automatiquement à partir de la description
    - Une annonce vendue doit être marquée comme indisponible
3. **Commande**
    - Une commande doit contenir au moins 1 ligne de commande
    - Le paiement doit être validé avant que la commande ne passe en préparation
    - Une commande livrée ne peut pas être annulée
4. **Review**
    - La note doit être comprise entre 1 et 5
    - Un utilisateur ne peut pas s'auto-évaluer
    - Un utilisateur doit avoir acheté auprès du vendeur pour laisser un avis (vérification par trigger)
5. **Signalement**
    - Un signalement doit avoir un motif et une description
    - Lors du traitement, une raison de décision doit être fournie

Règles de gestion

1. **Gestion des comptes**
    - Suppression RGPD : toutes les données personnelles doivent être supprimées (CASCADE)

    - Consentements : par défaut, seuls les cookies essentiels sont acceptés
    - Suspension : l'utilisateur ne peut plus se connecter mais ses données sont conservées
    - Bannissement : l'utilisateur ne peut plus se connecter et ses annonces sont supprimées

2. **Gestion des annonces**
    - Recherche sémantique : les embeddings permettent de trouver des annonces similaires
    - Une annonce vendue reste dans la base mais n'est plus disponible
    - Suppression : si l'annonce est dans une commande, elle ne peut être supprimée (intégrité référentielle)

3. **Gestion des paiements**
    - Les paiements sont gérés par un service tiers sécurisé
    - Une confirmation automatique est envoyée au client et au vendeur (trigger)
    - Un paiement validé ne peut pas être modifié (uniquement remboursé)

4. **Gestion des avis**
    - Les avis sont modérables par les administrateurs
    - Un avis signalé peut être supprimé après modération
    - La moyenne des notes d'un vendeur est calculée dynamiquement

5. **Administration**
    - Toutes les actions administratives sont loggées (traçabilité)
    - Les logs incluent : type d'action, cible, raison, IP, date
    - Les signalements doivent être traités ou rejetés (pas de suppression)

---

4. Conclusion

4.1 Synthèse

Tout au long de ce travail d'analyse, nous avons :

1. **Formalisé 35 exigences fonctionnelles** issues du cahier des charges, bien organisées en 8 grands domaines
2. **Créé 11 cas d'utilisation principaux** qui montrent qui fait quoi dans le système
3. **Détaillé plus de 20 scénarios** avec des diagrammes d'activités pour visualiser comment ça se passe concrètement
4. **Construit un modèle de données avec 14 entités** et toutes leurs relations

4.2 Couverture des exigences

Nous avons veillé à ne rien oublier du cahier des charges :

- **Exigences 1-5** : Consultation et recherche d'annonces
- **Exigences 6-11** : Gestion des comptes et RGPD
- **Exigences 14-16** : Paiement et confirmation
- **Exigences 17-19** : Navigation et avis
- **Exigences 20-24** : Gestion des annonces vendeur
- **Exigences 25-27** : Sécurité et ergonomie
- **Exigences 28-35** : Administration et modération

4.3 Points d'attention pour la suite

Pour la phase de conception technique, il faudra être particulièrement vigilant sur :

1. **La recherche sémantique** : mettre en place les embeddings avec PostgreSQL (extension pgvector), c'est technique
2. **La sécurité des paiements** : bien intégrer Stripe ou PayPal, c'est un point critique
3. **Le RGPD** : implémenter correctement le système de consentements et le droit à l'oubli
4. **Les performances** : bien optimiser les requêtes et le cache pour que ça reste fluide
5. **La modération** : créer un workflow efficace pour traiter les signalements

4.4 Ce qu'on peut en attendre

Le système OnlyShoes qu'on a modélisé pose de bonnes bases pour :

- Une marketplace spécialisée chaussures entre particuliers, simple et efficace
- Un système de notation qui permet de faire confiance aux vendeurs
- Une recherche qui combine filtres classiques et intelligence artificielle
- Des outils d'administration complets pour garder un contenu de qualité
- Un respect total du RGPD et de la vie privée des utilisateurs

---

Annexes

A. Glossaire

- **Annonce** : Produit (paire de chaussures) mis en vente par un vendeur
- **Embeddings** : Représentation vectorielle d'un texte permettant la recherche sémantique
- **RGPD** : Règlement Général sur la Protection des Données
- **Review** : Avis laissé par un acheteur sur une entité
- **Signalement** : Déclaration d'un contenu inapproprié (utilisateur, annonce ou avis)
- **Trigger** : Déclencheur automatique en base de données (ex: génération d'embeddings)
- **UUID** : Identifiant unique universel (format : xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx)

B. Références

- **Cahier des charges** : Document fourni avec 35 exigences fonctionnelles
- **UML** : Unified Modeling Language (langage de modélisation)
- **PlantUML** : Outil de génération de diagrammes UML
- **PostgreSQL** : Système de gestion de base de données relationnelle

- **CodeIgniter 4** : Framework PHP MVC

Document réalisé par : Groupe_1_03
Date de livraison : 9 janvier 2026
Version : 1.0